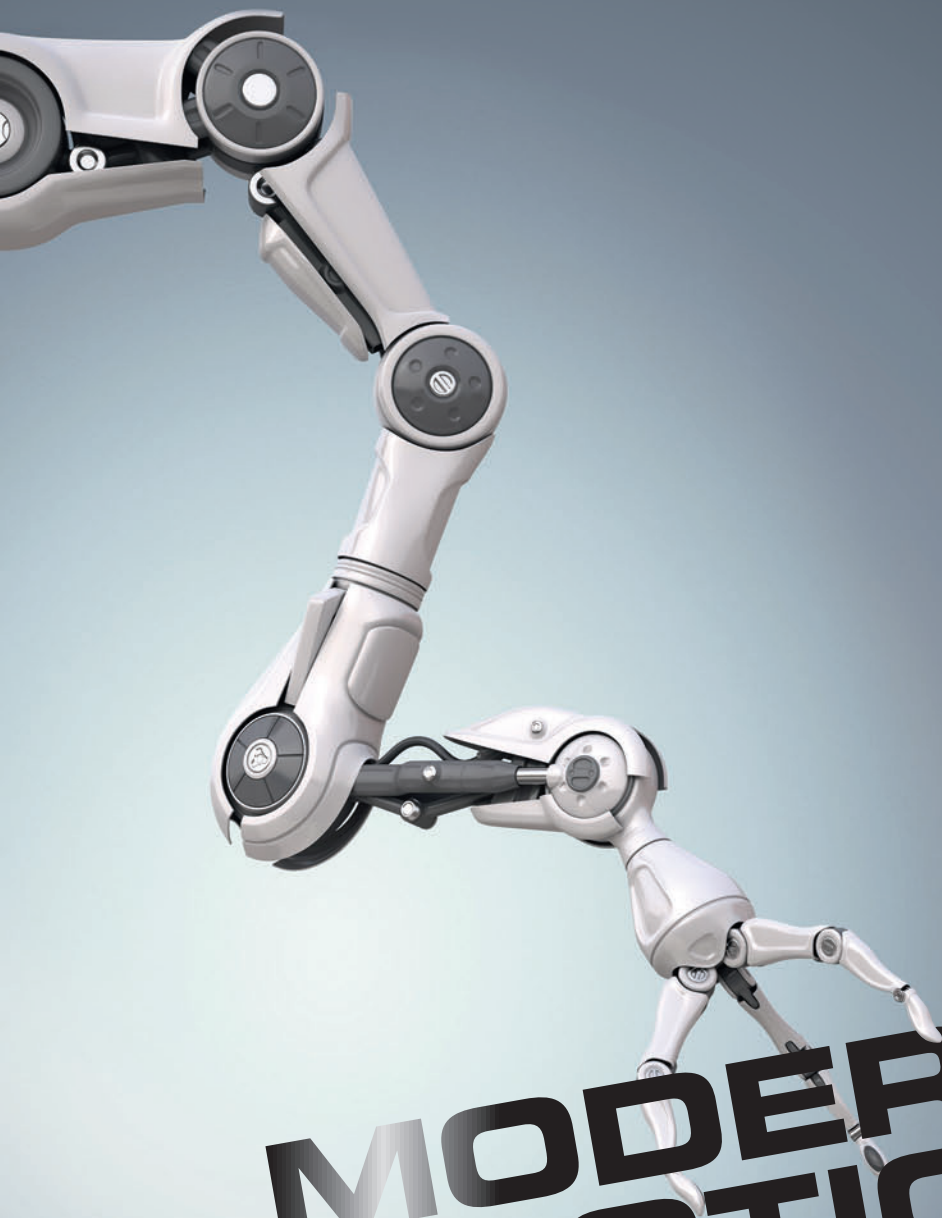




MODERN ROBOTICS

**MECHANICS, PLANNING,
AND CONTROL**

KEVIN M. LYNCH
FRANK C. PARK

Modern Robotics

Mechanics, Planning, and Control

This introduction to robotics offers a distinct and unified perspective of the mechanics, planning, and control of robots. It is ideal for self-learning, or for courses, as it assumes only freshman-level physics, ordinary differential equations, linear algebra, and a little bit of computing background. Modern Robotics:

- Presents the state-of-the-art screw-theoretic techniques capturing the most salient physical features of a robot in an intuitive geometrical way;
- Includes numerous exercises at the end of each chapter;
- Has accompanying, freely-downloadable software written to reinforce book concepts;
- Provides freely-downloadable video lectures aimed at changing the classroom experience, which students can watch in their own time, whilst class time is focused on collaborative problem-solving;
- Offers instructors the opportunity to design both one- and two-semester courses tailored to emphasize a range of topics, such as kinematics of robots and wheeled vehicles, kinematics and motion planning, mechanics of manipulation, and robot control;
- Can be used either with courses or for self-learning.

KEVIN M. LYNCH received his B.S.E. in Electrical Engineering from Princeton, New Jersey in 1989, and Ph.D. in Robotics from Carnegie Mellon University, Pennsylvania in 1996. He has been a faculty member at Northwestern University, Illinois since 1997 and has held visiting positions at California Institute of Technology, Carnegie Mellon University, Tsukuba University, Japan and Northeastern University in Shenyang, China. His research focuses on dynamics, motion planning and control for robot manipulation and locomotion; self-organizing multi-agent systems; and physically interacting human-robot systems. A Fellow of the Institute of Electrical and Electronics Engineers (IEEE), he also was the recipient of the IEEE Early Career Award in Robotics and Automation, Northwestern's Professorship of Teaching Excellence, and the Northwestern Teacher of the Year award in engineering. Currently he is Senior Editor of the IEEE Robotics and Automation Letters, and the incoming Editor-in-Chief of the IEEE International Conference on Robotics and Automation. This is his third book.

FRANK C. PARK received his B.S. in Electrical Engineering from Massachusetts Institute of Technology in 1985, and his Ph.D. in Applied Mathematics from Harvard University, Massachusetts in 1991. He has been on the faculty at University of California, Irvine and since 1995 he has been Professor of Mechanical and Aerospace Engineering at Seoul National University. His research interests are in robot mechanics, planning and control, vision and image processing, and related areas of applied mathematics. He has been an Institute of Electrical and Electronics Engineers (IEEE) Robotics and Automation Society Distinguished Lecturer and has held adjunct faculty positions at the Courant Institute of Mathematical Sciences, New York, the Interactive Computing Department at Georgia Institute of Technology and the Hong Kong University of Science and Technology Robotics Institute. He is a Fellow of the IEEE, Editor-in-Chief of the *IEEE Transactions on Robotics*, and developer of the EDX course Robot Mechanics and Control I, II.

Modern Robotics

Mechanics, Planning, and Control

KEVIN M. LYNCH

Northwestern University, Illinois

FRANK C. PARK

Seoul National University



CAMBRIDGE
UNIVERSITY PRESS

CAMBRIDGE

UNIVERSITY PRESS

University Printing House, Cambridge CB2 8BS, United Kingdom

One Liberty Plaza, 20th Floor, New York, NY 10006, USA

477 Williamstown Road, Port Melbourne, VIC 3207, Australia

4843/24, 2nd Floor, Ansari Road, Daryaganj, Delhi – 110002, India

79 Anson Road, #06–04/06, Singapore 079906

Cambridge University Press is part of the University of Cambridge.

It furthers the University's mission by disseminating knowledge in the pursuit of education, learning, and research at the highest international levels of excellence.

www.cambridge.org

Information on this title: www.cambridge.org/9781107156302

DOI: 10.1017/9781316661239

© Kevin M. Lynch and Frank C. Park 2017

This publication is in copyright. Subject to statutory exception and to the provisions of relevant collective licensing agreements, no reproduction of any part may take place without the written permission of Cambridge University Press.

First published 2017

Printed in the United Kingdom by TJ International Ltd., Padstow, Cornwall

A catalogue record for this publication is available from the British Library.

Library of Congress Cataloging-in-Publication Data

ISBN 978-1-107-15630-2 Hardback

ISBN 978-1-316-60984-2 Paperback

Cambridge University Press has no responsibility for the persistence or accuracy of URLs for external or third-party internet websites referred to in this publication and does not guarantee that any content on such websites is, or will remain, accurate or appropriate.

Contents

	<i>Foreword by Roger Brockett</i>	<i>page</i> xi
	<i>Foreword by Matthew Mason</i>	xiii
	<i>Preface</i>	xv
1	Preview	1
2	Configuration Space	10
	2.1 Degrees of Freedom of a Rigid Body	11
	2.2 Degrees of Freedom of a Robot	14
	2.3 Configuration Space: Topology and Representation	20
	2.4 Configuration and Velocity Constraints	25
	2.5 Task Space and Workspace	28
	2.6 Summary	32
	2.7 Notes and References	33
	2.8 Exercises	33
3	Rigid-Body Motions	50
	3.1 Rigid-Body Motions in the Plane	53
	3.2 Rotations and Angular Velocities	58
	3.3 Rigid-Body Motions and Twists	75
	3.4 Wrenches	92
	3.5 Summary	94
	3.6 Software	96
	3.7 Notes and References	97
	3.8 Exercises	98
4	Forward Kinematics	116
	4.1 Product of Exponentials Formula	119
	4.2 The Universal Robot Description Format	129
	4.3 Summary	134
	4.4 Software	135
	4.5 Notes and References	136
	4.6 Exercises	136

5	Velocity Kinematics and Statics	146
5.1	Manipulator Jacobian	152
5.2	Statics of Open Chains	162
5.3	Singularity Analysis	163
5.4	Manipulability	168
5.5	Summary	171
5.6	Software	172
5.7	Notes and References	172
5.8	Exercises	173
6	Inverse Kinematics	187
6.1	Analytic Inverse Kinematics	189
6.2	Numerical Inverse Kinematics	193
6.3	Inverse Velocity Kinematics	199
6.4	A Note on Closed Loops	200
6.5	Summary	201
6.6	Software	201
6.7	Notes and References	202
6.8	Exercises	202
7	Kinematics of Closed Chains	209
7.1	Inverse and Forward Kinematics	210
7.2	Differential Kinematics	215
7.3	Singularities	219
7.4	Summary	223
7.5	Notes and References	224
7.6	Exercises	225
8	Dynamics of Open Chains	231
8.1	Lagrangian Formulation	232
8.2	Dynamics of a Single Rigid Body	241
8.3	Newton–Euler Inverse Dynamics	248
8.4	Dynamic Equations in Closed Form	252
8.5	Forward Dynamics of Open Chains	255
8.6	Dynamics in the Task Space	256
8.7	Constrained Dynamics	257
8.8	Robot Dynamics in the URDF	259
8.9	Actuation, Gearing, and Friction	259
8.10	Summary	269
8.11	Software	273
8.12	Notes and References	274
8.13	Exercises	275

9	Trajectory Generation	278
9.1	Definitions	278
9.2	Point-to-Point Trajectories	279
9.3	Polynomial Via Point Trajectories	285
9.4	Time-Optimal Time Scaling	287
9.5	Summary	295
9.6	Software	296
9.7	Notes and References	297
9.8	Exercises	298
10	Motion Planning	302
10.1	Overview of Motion Planning	302
10.2	Foundations	306
10.3	Complete Path Planners	315
10.4	Grid Methods	316
10.5	Sampling Methods	323
10.6	Virtual Potential Fields	329
10.7	Nonlinear Optimization	336
10.8	Smoothing	337
10.9	Summary	338
10.10	Notes and References	340
10.11	Exercises	341
11	Robot Control	345
11.1	Control System Overview	346
11.2	Error Dynamics	346
11.3	Motion Control with Velocity Inputs	354
11.4	Motion Control with Torque or Force Inputs	360
11.5	Force Control	372
11.6	Hybrid Motion–Force Control	374
11.7	Impedance Control	378
11.8	Low-Level Joint Force–Torque Control	381
11.9	Other Topics	383
11.10	Summary	385
11.11	Software	387
11.12	Notes and References	388
11.13	Exercises	389
12	Grasping and Manipulation	396
12.1	Contact Kinematics	397
12.2	Contact Forces and Friction	415
12.3	Manipulation	425
12.4	Summary	431
12.5	Notes and References	432
12.6	Exercises	433

13	Wheeled Mobile Robots	441
13.1	Types of Wheeled Mobile Robots	441
13.2	Omnidirectional Wheeled Mobile Robots	443
13.3	Nonholonomic Wheeled Mobile Robots	448
13.4	Odometry	469
13.5	Mobile Manipulation	471
13.6	Summary	474
13.7	Notes and References	476
13.8	Exercises	477
A	Summary of Useful Formulas	486
B	Other Representations of Rotations	493
C	Denavit–Hartenberg Parameters	502
D	Optimization and Lagrange Multipliers	512
	Bibliography	515
	Index	524

Foreword by Roger Brockett

In the 1870s, Felix Klein was developing his far-reaching Erlangen Program, which cemented the relationship between geometry and group theoretic ideas. With Sophus Lie's nearly simultaneous development of a theory of continuous (Lie) groups, important new tools involving infinitesimal analysis based on Lie algebraic ideas became available for the study of a very wide range of geometric problems. Even today, the thinking behind these ideas continues to guide developments in important areas of mathematics. Kinematic mechanisms are, of course, more than just geometry; they need to accelerate, avoid collisions, etc., but first of all they are geometrical objects and the ideas of Klein and Lie apply. The groups of rigid motions in two or three dimensions, as they appear in robotics, are important examples in the work of Klein and Lie.

In the mathematics literature the representation of elements of a Lie group in terms of exponentials usually takes one of two different forms. These are known as exponential coordinates of the first kind and exponential coordinates of the second kind. For the first kind one has $X = e^{(A_1x_1 + A_2x_2 \dots)}$. For the second kind this is replaced by $X = e^{A_1x_1}e^{A_2x_2} \dots$. Up until now, the first choice has found little utility in the study of kinematics whereas the second choice, a special case having already shown up in Euler parametrizations of the orthogonal group, turns out to be remarkably well-suited for the description of open kinematic chains consisting of the concatenation of single degree of freedom links. This is all nicely explained in Chapter 4 of this book. Together with the fact that $Pe^AP^{-1} = e^{PAP^{-1}}$, the second form allows one to express a wide variety of kinematic problems very succinctly. From a historical perspective, the use of the product of exponentials to represent robotic movement, as the authors have done here, can be seen as illustrating the practical utility of the 150-year-old ideas of the geometers Klein and Lie.

In 1983 I was invited to speak at the triennial Mathematical Theory of Networks and Systems Conference in Beer Sheva, Israel, and after a little thought I decided to try to explain something about what my recent experiences had taught me. By then I had some experience in teaching a robotics course that discussed kinematics, including the use of the product of exponentials representation of kinematic chains. From the 1960s onward e^{At} had played a central role in system theory and signal processing, so at this conference a familiarity, even an affection, for the matrix exponential could be counted on. Given this, it was

natural for me to pick something e^{Ax} -related for the talk. Although I had no reason to think that there would be many in the audience with an interest in kinematics, I still hoped that I could say something interesting and maybe even inspire further developments. The result was the paper referred to in the preface that follows.

In this book, Frank and Kevin have provided a wonderfully clear and patient explanation of their subject. They translate the foundation laid out by Klein and Lie 150 years ago to the modern practice of robotics, at a level appropriate for undergraduate engineers. After an elegant discussion of the fundamental properties of configuration spaces, they introduce the Lie group representations of rigid-body configurations, and the corresponding representations of velocities and forces, used throughout the book. This consistent perspective is carried through foundational robotics topics including the forward, inverse, and differential kinematics of open chains, robot dynamics, trajectory generation, robot control, and more specialized topics such as the kinematics of closed chains, motion planning, robot manipulation, planning and control for wheeled mobile robots, and the control of mobile manipulators.

I am confident that this book will be a valuable resource for a generation of students and practitioners of robotics.

Roger Brockett
Cambridge, Massachusetts, USA
November 2016

Foreword by Matthew Mason

Robotics is about turning ideas into action. Somehow, robots turn abstract goals into physical action: sending power to motors, monitoring motions, and guiding things towards the goal. Every human can perform this trick, but it is nonetheless so intriguing that it has captivated philosophers and scientists, including Descartes and many others.

What is the secret? Did some roboticist have a eureka moment? Did some pair of teenage entrepreneurs hit on the key idea in their garage? To the contrary, it is not a single idea. It is a substantial body of scientific and engineering results, accumulated over centuries. It draws primarily from mathematics, physics, mechanical engineering, electrical engineering and computer science, but also from philosophy, psychology, biology and other fields.

Robotics is the gathering place of these ideas. Robotics provides motivation. Robotics tests ideas and steers continuing research. Finally, robotics is the proof. Observing a robot's behavior is the nearly compelling proof that machines can be aware of their surroundings, can develop meaningful goals, and can act effectively to accomplish those goals. The same principles apply to a thermostat or a fly-ball governor, but few are persuaded by watching a thermostat. Nearly all are persuaded by watching a robot soccer team.

The heart of robotics is motion – controlled programmable motion – which brings us to the present text. *Modern Robotics* imparts the most important insights of robotics: the nature of motion, the motions available to rigid bodies, the use of kinematic constraint to organize motions, the mechanisms that enable general programmable motion, the static and dynamic character of mechanisms, and the challenges and approaches to control, programming, and planning motions. *Modern Robotics* presents this material with a clarity that makes it accessible to undergraduate students. It is distinguished from other undergraduate texts in two important ways.

First, in addressing rigid-body motion, *Modern Robotics* presents not only the classical geometrical underpinnings and representations, but also their expression using modern matrix exponentials, and the connection to Lie algebras. The rewards to the students are two-fold: a deeper understanding of motion, and better practical tools.

Second, *Modern Robotics* goes beyond a focus on robot mechanisms to address the interaction with objects in the surrounding world. When robots make contact

with the real world, the result is an *ad hoc* kinematic mechanism, with associated statics and dynamics. The mechanism includes kinematic loops, unactuated joints, and nonholonomic constraints, all of which will be familiar concepts to students of *Modern Robotics*.

Even if this is the only robotics course students take, it will enable them to analyze, control, and program a wide range of physical systems. With its introduction to the mechanics of physical interaction, *Modern Robotics* is also an excellent beginning for the student who intends to continue with advanced courses or with original research in robotics.

Matthew T. Mason
Pittsburgh, Pennsylvania, USA
November 2016

Preface

It was at the IEEE International Conference on Robotics and Automation in Pasadena in 2008 when, over a beer, we decided to write an undergraduate textbook on robotics. Since 1996, Frank had been teaching robot kinematics to Seoul National University undergraduates using his own lecture notes; by 2008 these notes had evolved to the kernel around which this book was written. Kevin had been teaching his introductory robotics class at Northwestern University from his own set of notes, with content drawn from an eclectic collection of papers and books.

We believe that there is a distinct and unifying perspective to the mechanics, planning, and control for robots that is lost if these subjects are studied independently or as part of other more traditional subjects. At the 2008 meeting, we noted the lack of a textbook that (a) treated these topics in a unified way, with plenty of exercises and figures, and (b), most importantly, was written at a level appropriate for a first robotics course for undergraduates with only freshman-level physics, ordinary differential equations, linear algebra, and a little bit of computing background. We decided that the only sensible recourse was to write such a book ourselves. (We didn't know then that it would take us more than eight years to finish the project!)

A second motivation for this book, and one that we believe sets it apart from other introductory treatments on robotics, is its emphasis on modern geometric techniques. Often the most salient physical features of a robot are best captured by a geometric description. The advantages of the geometric approach have been recognized for quite some time by practitioners of classical screw theory. What has made these tools largely inaccessible to undergraduates – the primary target audience for this book – is that they require an entirely new language of concepts and constructs (screws, twists, wrenches, reciprocity, transversality, conjugacy, etc.), and their often obscure rules for manipulation and transformation. However, the mostly algebraic alternatives to screw theory often mean that students end up buried in the details of calculation, losing the simple and elegant geometric interpretation that lies at the heart of what they are calculating.

The breakthrough that made the techniques of classical screw theory accessible to a more general audience arrived in the early 1980s, when Roger Brockett showed how to describe kinematic chains mathematically in terms of the Lie group structure of rigid-body motions (Brockett, 1983b). This discovery allowed

one, among other things, to re-invent screw theory simply by appealing to basic linear algebra and linear differential equations. With this “modern screw theory” the powerful tools of modern differential geometry can be brought to bear on a wide-ranging collection of robotics problems, some of which we explore here, and others of which are covered in the excellent but more advanced graduate textbook by Murray et al. (1994).

As the title indicates, this book covers what we feel to be the fundamentals of robot mechanics, together with the basics of planning and control. A thorough treatment of all the chapters would likely take two semesters, particularly when coupled with programming assignments or experiments with robots. The contents of Chapters 2–6 constitute the minimum essentials, and these topics should probably be covered in sequence.

The instructor can then selectively choose content from the remaining chapters. At Seoul National University, the undergraduate course M2794.0027 Introduction to Robotics covers, in one semester, Chapters 2–7 and parts of Chapters 10–12. At Northwestern, ME 449 Robotic Manipulation covers, in an 11-week quarter, Chapters 2–6 and 8, and then touches on different topics in Chapters 9–13 depending on the interests of the students and instructor. A course focusing on the kinematics of robot arms and wheeled vehicles could cover Chapters 2–7 and 13, while one on kinematics and motion planning could additionally include Chapters 9 and 10. A course on the mechanics of manipulation would cover Chapters 2–6, 8, and 12, while another on robot control would cover Chapters 2–6, 8, 9, and 11. If the instructor prefers to avoid dynamics (Chapter 8), the basics of robot control (Chapters 11 and 13) can be covered by assuming that the velocity at each actuator is controlled, not the forces and torques. A course focusing only on motion planning could cover Chapters 2 and 3, Chapter 10 in depth (possibly supplemented by research papers or other references cited in that chapter), and Chapter 13.

To help the instructor choose which topics to teach and to help the student keep track of what she has learned, we have included summaries at the ends of chapters and a summary of important notation and formulas used throughout the book in Appendix A. For those whose primary interest in this text is as an introductory reference, we have attempted to provide a reasonably comprehensive, though by no means exhaustive, set of references and bibliographic notes at the end of each chapter. Some of the exercises provided at the end of each chapter extend the basic results covered in the book and, for those who wish to probe further, these should be of interest in their own right. Some of the more advanced material in the book can be used to support independent study projects.

Another important component of the book is the software, which is written to reinforce the concepts in the book and to make the formulas operational. The software was developed primarily by Kevin’s ME 449 students at Northwestern and is freely downloadable from <http://modernrobotics.org>. Video lectures that accompany the textbook will also be available at the website. The intention

of the video content is to help the instructor to “flip” the classroom: students watch brief lectures on their own time, rewinding as needed, and class time is focused more on the collaborative problem-solving that has traditionally occurred between classes. This way, the professor is present when the students are applying the material and discovering the gaps in their understanding; this creates the opportunity for interactive mini-lectures addressing the concepts that need most reinforcing. We believe that the added value of the professor is greatest in this interactive role, not in delivering a lecture in the same way that it was delivered the previous year. This approach has worked well for Kevin’s introduction to mechatronics course, <http://nu32.org>.

The video content is generated using the Lightboard, <http://lightboard.info>, created by Michael Peshkin at Northwestern University. We thank him for sharing this convenient and effective tool for creating instructional videos.

We have also found the V-REP robot simulation software to be a valuable supplement to the book and its software. This simulation software allows students to explore interactively the kinematics of robot arms and mobile manipulators and to animate trajectories that are the result of exercises on kinematics, dynamics, and control.

While this book presents our own perspective on how to introduce the fundamental topics in first courses on robot mechanics, planning, and control, we acknowledge the excellent textbooks that already exist and that have served our field well. Among these, we would like to mention as particularly influential the books by Murray et al. (1994), Craig (2004), Spong et al. (2005), Siciliano et al. (2009), Mason (2001), and Corke (2017), and the motion planning books by Latombe (1991), LaValle (2006), and Choset et al. (2005). In addition, the *Handbook of Robotics* (Siciliano and Khatib, 2016), with a multimedia extension edited by Kröger (<http://handbookofrobotics.org>), is a landmark in our field, collecting the perspectives of hundreds of leading researchers on a huge variety of topics relevant to modern robotics.

It is our pleasure to acknowledge the many people who have been the sources of help and inspiration in writing this book. In particular, we would like to thank our Ph.D. advisors, Roger Brockett and Matt Mason. Brockett laid down much of the foundation for the geometric approach to robotics employed in this book. Mason’s pioneering contributions to analysis and planning for manipulation form a cornerstone of modern robotics. We also thank the many students who provided feedback on various versions of this material, in M2794.0027 at Seoul National University and in ME 449 at Northwestern University. In particular, Frank would like to thank Seunghyeon Kim, Keunjun Choi, Jisoo Hong, Jinkyu Kim, Youngsuk Hong, Wooyoung Kim, Cheongjae Jang, Taeyoon Lee, Soocheol Noh, Kyumin Park, Seongjae Jeong, Sukho Yoon, Jaewoon Kwen, Jinyhyuk Park, and Jihoon Song, as well as Jim Bobrow and Scott Ploen from his time at UC Irvine. Kevin would like to thank Matt Elwin, Sherif Mostafa, Nelson Rosa, Jarvis Schultz, Jian Shi, Mikhail Todes, Huan Weng, and Zack Woodruff.

We would also like to thank Susan Parkinson and David Tranah at Cambridge

University Press for their diligence and expertise in matters to do with copy-editing, proof-reading, and layout.

Finally, and most importantly, we thank our wives and families, for putting up with our late nights and our general unavailability, and for supporting us as we made the final push to finish the book. Without the love and support of Hyunmee, Shiyeon, and Soonkyu (Frank's family) and Yuko, Erin, and Patrick (Kevin's family), this book would not exist. We dedicate this book to them.

Kevin M. Lynch
Evanston, Illinois, USA

Frank C. Park
Seoul, Korea

November 2016

Publication note.

The authors consider themselves to be equal contributors to this book. The author order is alphabetical.

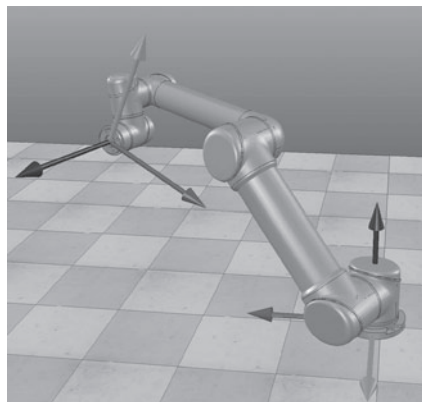
1 Preview

As an academic discipline, robotics is a relatively young field with highly ambitious goals, the ultimate one being the creation of machines that can behave and think like humans. This attempt to create intelligent machines naturally leads us first to examine ourselves – to ask, for example, why our bodies are designed the way they are, how our limbs are coordinated, and how we learn and perform complex tasks. The sense that the fundamental questions in robotics are ultimately questions about ourselves is part of what makes robotics such a fascinating and engaging endeavor.

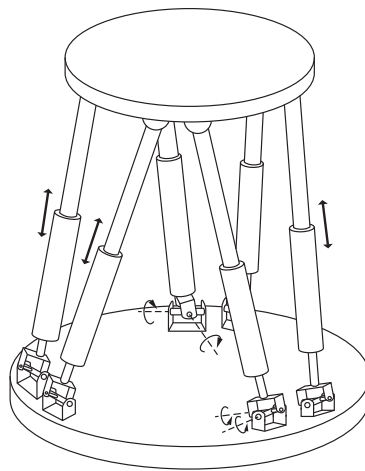
Our focus in this book is on mechanics, planning, and control for **robot mechanisms**. Robot arms are one familiar example. So are wheeled vehicles, as are robot arms mounted on wheeled vehicles. Basically, a mechanism is constructed by connecting rigid bodies, called **links**, together by means of **joints**, so that relative motion between adjacent links becomes possible. **Actuation** of the joints, typically by electric motors, then causes the robot to move and exert forces in desired ways.

The links of a robot mechanism can be arranged in serial fashion, like the familiar open-chain arm shown in Figure 1.1(a). Robot mechanisms can also have links that form closed loops, such as the Stewart–Gough platform shown in Figure 1.1(b). In the case of an open chain, all the joints are actuated, while in the case of mechanisms with closed loops, only a subset of the joints may be actuated.

Let us examine more closely the current technology behind robot mechanisms. The links are moved by actuators, which typically are electrically driven (e.g., by DC or AC motors, stepper motors, or shape memory alloys) but can also be driven by pneumatic or hydraulic cylinders. In the case of rotating electric motors, these would ideally be lightweight, operate at relatively low rotational speeds (e.g., in the range of hundreds of RPM), and be able to generate large forces and torques. Since most currently available motors operate at low torques and at up to thousands of RPM, speed reduction and torque amplification are required. Examples of such transmissions or transformers include gears, cable drives, belts and pulleys, and chains and sprockets. These speed-reduction devices should have zero or low slippage and **backlash** (defined as the amount of rotation available at the output of the speed-reduction device without motion at



(a) An open-chain industrial manipulator, visualized in V-REP (Rohmer et al., 2013).



(b) Stewart–Gough platform. Closed loops are formed from the base platform, through the legs, through the top platform, and through the legs back to the base platform.

Figure 1.1 Open-chain and closed-chain robot mechanisms.

the input). Brakes may also be attached to stop the robot quickly or to maintain a stationary posture.

Robots are also equipped with sensors to measure the motion at the joints. For both revolute and prismatic joints, encoders, potentiometers, or resolvers measure the displacement and sometimes tachometers are used to measure velocity. Forces and torques at the joints or at the end-effector of the robot can be measured using various types of force–torque sensors. Additional sensors may be used to help localize objects or the robot itself, such as vision-only cameras, RGB-D cameras which measure the color (RGB) and depth (D) to each pixel, laser range finders, and various types of acoustic sensor.

The study of robotics often includes artificial intelligence and computer perception, but an essential feature of any robot is that it moves in the physical world. Therefore, this book, which is intended to support a first course in robotics for undergraduates and graduate students, focuses on mechanics, motion planning, and control of robot mechanisms.

In the rest of this chapter we provide a preview of the rest of the book.

Chapter 2: Configuration Space

As mentioned above, at its most basic level a robot consists of rigid bodies connected by joints, with the joints driven by actuators. In practice the links may not be completely rigid, and the joints may be affected by factors such as

elasticity, backlash, friction, and hysteresis. In this book we ignore these effects for the most part and assume that all links are rigid.

With this assumption, Chapter 2 focuses on representing the **configuration** of a robot system, which is a specification of the position of every point of the robot. Since the robot consists of a collection of rigid bodies connected by joints, our study begins with understanding the configuration of a rigid body. We see that the configuration of a rigid body in the plane can be described using three variables (two for the position and one for the orientation) and the configuration of a rigid body in space can be described using six variables (three for the position and three for the orientation). The number of variables is the number of **degrees of freedom** (dof) of the rigid body. It is also the dimension of the **configuration space**, the space of all configurations of the body.

The dof of a robot, and hence the dimension of its configuration space, is the sum of the dof of its rigid bodies minus the number of constraints on the motion of those rigid bodies provided by the joints. For example, the two most popular joints, revolute (rotational) and prismatic (translational) joints, allow only one motion freedom between the two bodies they connect. Therefore a revolute or prismatic joint can be thought of as providing five constraints on the motion of one spatial rigid body relative to another. Knowing the dof of a rigid body and the number of constraints provided by joints, we can derive **Grübler's formula** for calculating the dof of general robot mechanisms. For **open-chain** robots such as the industrial manipulator of Figure 1.1(a), each joint is independently actuated and the dof is simply the sum of the freedoms provided by each joint. For **closed chains** like the Stewart–Gough platform in Figure 1.1(b), Grübler's formula is a convenient way to calculate a lower bound on the dof. Unlike open-chain robots, some joints of closed chains are not actuated.

Apart from calculating the dof, other configuration space concepts of interest include the **topology** (or “shape”) of the configuration space and its **representation**. Two configuration spaces of the same dimension may have different shapes, just like a two-dimensional plane has a different shape from the two-dimensional surface of a sphere. These differences become important when determining how to represent the space. The surface of a unit sphere, for example, could be represented using a minimal number of coordinates, such as latitude and longitude, or it could be represented by three numbers (x, y, z) subject to the constraint $x^2 + y^2 + z^2 = 1$. The former is an **explicit parametrization** of the space and the latter is an **implicit parametrization** of the space. Each type of representation has its advantages, but in this book we will use implicit representations of configurations of rigid bodies.

A robot arm is typically equipped with a hand or gripper, more generally called an **end-effector**, which interacts with objects in the surrounding world. To accomplish a task such as picking up an object, we are concerned with the configuration of a reference frame rigidly attached to the end-effector, and not necessarily the configuration of the entire arm. We call the space of positions and orientations of the end-effector frame the **task space** and note that there is

not a one-to-one mapping between the robot's configuration space and the task space. The **workspace** is defined to be the subset of the task space that the end-effector frame can reach.

Chapter 3: Rigid-Body Motions

This chapter addresses the problem of how to describe mathematically the motion of a rigid body moving in three-dimensional physical space. One convenient way is to attach a reference frame to the rigid body and to develop a way to quantitatively describe the frame's position and orientation as it moves. As a first step, we introduce a 3×3 matrix representation for describing a frame's orientation; such a matrix is referred to as a **rotation matrix**.

A rotation matrix is parametrized by three independent coordinates. The most natural and intuitive way to visualize a rotation matrix is in terms of its **exponential coordinate** representation. That is, given a rotation matrix R , there exists some unit vector $\hat{\omega} \in \mathbb{R}^3$ and angle $\theta \in [0, \pi]$ such that the rotation matrix can be obtained by rotating the identity frame (that is, the frame corresponding to the identity matrix) about $\hat{\omega}$ by θ . The exponential coordinates are defined as $\omega = \hat{\omega}\theta \in \mathbb{R}^3$, which is a three-parameter representation. There are several other well-known coordinate representations, e.g., Euler angles, Cayley–Rodrigues parameters, and unit quaternions, which are discussed in Appendix B.

Another reason for focusing on the exponential description of rotations is that they lead directly to the exponential description of rigid-body motions. The latter can be viewed as a modern geometric interpretation of classical screw theory. Keeping the classical terminology as much as possible, we cover in detail the linear algebraic constructs of screw theory, including the unified description of linear and angular velocities as six-dimensional **twists** (also known as **spatial velocities**), and an analogous description of three-dimensional forces and moments as six-dimensional **wrenches** (also known as **spatial forces**).

Chapter 4: Forward Kinematics

For an open chain, the position and orientation of the end-effector are uniquely determined from the joint positions. The **forward kinematics** problem is to find the position and orientation of the reference frame attached to the end-effector given the set of joint positions. In this chapter we present the **product of exponentials (PoE)** formula describing the forward kinematics of open chains. As the name implies, the PoE formula is directly derived from the exponential coordinate representation for rigid-body motions. Aside from providing an intuitive and easily visualizable interpretation of the exponential coordinates as the twists of the joint axes, the PoE formula offers other advantages, like eliminating the need for link frames (only the base frame and end-effector frame are required, and these can be chosen arbitrarily).

In Appendix C we also present the Denavit–Hartenberg (D–H) representation

for forward kinematics. The D–H representation uses fewer parameters but requires that reference frames be attached to each link following special rules of assignment, which can be cumbersome. Details of the transformation from the D–H to the PoE representation are also provided in Appendix C.

Chapter 5: Velocity Kinematics and Statics

Velocity kinematics refers to the relationship between the joint linear and angular velocities and those of the end-effector frame. Central to velocity kinematics is the **Jacobian** of the forward kinematics. By multiplying the vector of joint-velocity rates by this configuration-dependent matrix, the twist of the end-effector frame can be obtained for any given robot configuration. **Kinematic singularities**, which are configurations in which the end-effector frame loses the ability to move or rotate in one or more directions, correspond to those configurations at which the Jacobian matrix fails to have maximal rank. The **manipulability ellipsoid**, whose shape indicates the ease with which the robot can move in various directions, is also derived from the Jacobian.

Finally, the Jacobian is also central to static force analysis. In static equilibrium settings, the Jacobian is used to determine what forces and torques need to be exerted at the joints in order for the end-effector to apply a desired wrench.

The definition of the Jacobian depends on the representation of the end-effector velocity, and our preferred representation of the end-effector velocity is as a six-dimensional twist. We touch briefly on other representations of the end-effector velocity and their corresponding Jacobians.

Chapter 6: Inverse Kinematics

The **inverse kinematics** problem is to determine the set of joint positions that achieves a desired end-effector configuration. For open-chain robots, the inverse kinematics is in general more involved than the forward kinematics: for a given set of joint positions there usually exists a unique end-effector position and orientation but, for a particular end-effector position and orientation, there may exist multiple solutions to the joint positions, or no solution at all.

In this chapter we first examine a popular class of six-dof open-chain structures whose inverse kinematics admits a closed-form analytic solution. Iterative numerical algorithms are then derived for solving the inverse kinematics of general open chains by taking advantage of the inverse of the Jacobian. If the open-chain robot is **kinematically redundant**, meaning that it has more joints than the dimension of the task space, then we use the pseudoinverse of the Jacobian.

Chapter 7: Kinematics of Closed Chains

While open chains have unique forward kinematics solutions, closed chains often have multiple forward kinematics solutions, and sometimes even multiple solu-

tions for the inverse kinematics as well. Also, because closed chains possess both actuated and passive joints, the kinematic singularity analysis of closed chains presents subtleties not encountered in open chains. In this chapter we study the basic concepts and tools for the kinematic analysis of closed chains. We begin with a detailed case study of mechanisms such as the planar five-bar linkage and the Stewart–Gough platform. These results are then generalized into a systematic methodology for the kinematic analysis of more general closed chains.

Chapter 8: Dynamics of Open Chains

Dynamics is the study of motion taking into account the forces and torques that cause it. In this chapter we study the dynamics of open-chain robots. In analogy to the notions of a robot’s forward and inverse kinematics, the **forward dynamics** problem is to determine the resulting joint accelerations for a given set of joint forces and torques. The **inverse dynamics** problem is to determine the input joint torques and forces needed for desired joint accelerations. The dynamic equations relating the forces and torques to the motion of the robot’s links are given by a set of second-order ordinary differential equations.

The dynamics for an open-chain robot can be derived using one of two approaches. In the Lagrangian approach, first a set of coordinates – referred to as generalized coordinates in the classical dynamics literature – is chosen to parametrize the configuration space. The sum of the potential and kinetic energies of the robot’s links are then expressed in terms of the generalized coordinates and their time derivatives. These are then substituted into the **Euler–Lagrange equations**, which then lead to a set of second-order differential equations for the dynamics, expressed in the chosen coordinates for the configuration space.

The **Newton–Euler** approach builds on the generalization of $f = ma$, i.e., the equations governing the acceleration of a rigid body given the wrench acting on it. Given the joint variables and their time derivatives, the Newton–Euler approach to inverse dynamics is: to propagate the link velocities and accelerations outward from the proximal link to the distal link, in order to determine the velocity and acceleration of each link; to use the equations of motion for a rigid body to calculate the wrench (and therefore the joint force or torque) that must be acting on the outermost link; and to proceed along the links back toward the base of the robot, calculating the joint forces or torques needed to create the motion of each link and to support the wrench transmitted to the distal links. Because of the open-chain structure, the dynamics can be formulated recursively.

In this chapter we examine both approaches to deriving a robot’s dynamic equations. Recursive algorithms for both the forward and inverse dynamics, as well as analytical formulations of the dynamic equations, are presented.

Chapter 9: Trajectory Generation

What sets a robot apart from an automated machine is that it should be easily reprogrammable for different tasks. Different tasks require different motions, and it would be unreasonable to expect the user to specify the entire time-history of each joint for every task; clearly it would be desirable for the robot's control computer to "fill in the details" from a small set of task input data.

This chapter is concerned with the automatic generation of joint trajectories from this set of task input data. Formally, a trajectory consists of a **path**, which is a purely geometric description of the sequence of configurations achieved by a robot, and a **time scaling**, which specifies the times at which those configurations are reached.

Often the input task data is given in the form of an ordered set of joint values, called control points, together with a corresponding set of control times. On the basis of this data the trajectory generation algorithm produces a trajectory for each joint which satisfies various user-supplied conditions. In this chapter we focus on three cases: (i) point-to-point straight-line trajectories in both joint space and task space; (ii) smooth trajectories passing through a sequence of timed "via points"; and (iii) time-optimal trajectories along specified paths, subject to the robot's dynamics and actuator limits. Finding paths that avoid collisions is the subject of the next chapter on motion planning.

Chapter 10: Motion Planning

This chapter addresses the problem of finding a collision-free motion for a robot through a cluttered workspace, while avoiding joint limits, actuator limits, and other physical constraints imposed on the robot. The **path planning** problem is a subproblem of the general motion planning problem that is concerned with finding a collision-free path between a start and goal configuration, usually without regard to the dynamics, the duration of the motion, or other constraints on the motion or control inputs.

There is no single planner applicable to all motion planning problems. In this chapter we consider three basic approaches: grid-based methods, sampling methods, and methods based on virtual potential fields.

Chapter 11: Robot Control

A robot arm can exhibit a number of different behaviors depending on the task and its environment. It can act as a source of programmed motions for tasks such as moving an object from one place to another, or tracing a trajectory for manufacturing applications. It can act as a source of forces, for example when grinding or polishing a workpiece. In tasks such as writing on a chalkboard, it must control forces in some directions (the force pressing the chalk against the board) and motions in other directions (the motion in the plane of the board).

In certain applications, e.g., haptic displays, we may want the robot to act like a programmable spring, damper, or mass, by controlling its position, velocity, or acceleration in response to forces applied to it.

In each of these cases, it is the job of the robot controller to convert the task specification to forces and torques at the actuators. Control strategies to achieve the behaviors described above are known as **motion** (or **position**) **control**, **force control**, **hybrid motion–force control**, and **impedance control**. Which of these behaviors is appropriate depends on both the task and the environment. For example, a force-control goal makes sense when the end-effector is in contact with something, but not when it is moving in free space. We also have a fundamental constraint imposed by the mechanics, irrespective of the environment: the robot cannot independently control both motions and forces in the same direction. If the robot imposes a motion then the environment determines the force, and vice versa.

Most robots are driven by actuators that apply a force or torque to each joint. Hence, precisely controlling a robot requires an understanding of the relationship between the joint forces and torques and the motion of the robot; this is the domain of dynamics. Even for simple robots, however, the dynamic equations are complex and dependent on a precise knowledge of the mass and inertia of each link, which may not be readily available. Even if it were, the dynamic equations would still not reflect physical phenomena such as friction, elasticity, backlash, and hysteresis.

Most practical control schemes compensate for these uncertainties by using **feedback control**. After examining the performance limits of feedback control without a dynamic model of the robot, we study motion control algorithms, such as **computed torque control**, that combine approximate dynamic modeling with feedback control. The basic lessons learned for robot motion control are then applied to force control, hybrid motion–force control, and impedance control.

Chapter 12: Grasping and Manipulation

The focus of earlier chapters is on characterizing, planning, and controlling the motion of the robot itself. To do useful work, the robot must be capable of manipulating objects in its environment. In this chapter we model the contact between the robot and an object, specifically the constraints on the object motion imposed by a contact and the forces that can be transmitted through a frictional contact. With these models we study the problem of choosing contacts to immobilize an object by **form closure** and **force closure** grasping. We also apply contact modeling to manipulation problems other than grasping, such as pushing an object, carrying an object dynamically, and testing the stability of a structure.

Chapter 13: Wheeled Mobile Robots

The final chapter addresses the kinematics, motion planning, and control of wheeled mobile robots and of wheeled mobile robots equipped with robot arms. A mobile robot can use specially designed **omniwheels** or **mecanum wheels** to achieve omnidirectional motion, including spinning in place or translating in any direction. Many mobile bases, however, such as cars and differential-drive robots, use more typical wheels, which do not slip sideways. These no-slip constraints are fundamentally different from the loop-closure constraints found in closed chains; the latter are **holonomic**, meaning that they are configuration constraints, while the former are **nonholonomic**, meaning that the velocity constraints cannot be integrated to become equivalent configuration constraints.

Because of the different properties of omnidirectional mobile robots versus nonholonomic mobile robots, we consider their kinematic modeling, motion planning, and control separately. In particular, the motion planning and control of nonholonomic mobile robots is more challenging than for omnidirectional mobile robots.

Once we have derived their kinematic models, we show that the **odometry** problem – the estimation of the chassis configuration based on wheel encoder data – can be solved in the same way for both types of mobile robots. Similarly, for mobile manipulators consisting of a wheeled base and a robot arm, we show that feedback control for **mobile manipulation** (controlling the motion of the end-effector using the arm joints and wheels) is the same for both types of mobile robots. The fundamental object in mobile manipulation is the Jacobian mapping joint rates and wheel velocities to end-effector twists.

Each chapter concludes with a summary of important concepts from the chapter, and Appendix A compiles some of the most used equations into a handy reference. Videos supporting the book can be found at the book's website, <http://modernrobotics.org>. Some chapters have associated software, downloadable from the website. The software is meant to be neither maximally robust nor efficient but to be readable and to reinforce the concepts in the book. You are encouraged to read the software, not just use it, to cement your understanding of the material. Each function contains a sample usage in the comments. The software package may grow over time, but the core functions are documented in the chapters themselves.

2 Configuration Space

A robot is mechanically constructed by connecting a set of bodies, called **links**, to each other using various types of **joints**. **Actuators**, such as electric motors, deliver forces or torques that cause the robot's links to move. Usually an **end-effector**, such as a gripper or hand for grasping and manipulating objects, is attached to a specific link. All the robots considered in this book have links that can be modeled as rigid bodies.

Perhaps the most fundamental question one can ask about a robot is, where is it? The answer is given by the robot's **configuration**: a specification of the positions of all points of the robot. Since the robot's links are rigid and of a known shape,¹ only a few numbers are needed to represent its configuration. For example, the configuration of a door can be represented by a single number, the angle θ about its hinge. The configuration of a point on a plane can be described by two coordinates, (x, y) . The configuration of a coin lying heads up on a flat table can be described by three coordinates: two coordinates (x, y) that specify the location of a particular point on the coin, and one coordinate (θ) that specifies the coin's orientation. (See Figure 2.1).

The above coordinates all take values over a continuous range of real numbers. The number of **degrees of freedom (dof)** of a robot is the smallest number of real-valued coordinates needed to represent its configuration. In the example above, the door has one degree of freedom. The coin lying heads up on a table has three degrees of freedom. Even if the coin could lie either heads up or tails up, its configuration space still would have only three degrees of freedom; a fourth variable, representing which side of the coin faces up, takes values in the discrete set {heads, tails}, and not over a continuous range of real values like the other three coordinates.

Definition 2.1 The **configuration** of a robot is a complete specification of the position of every point of the robot. The minimum number n of real-valued coordinates needed to represent the configuration is the number of **degrees of freedom (dof)** of the robot. The n -dimensional space containing all possible configurations of the robot is called the **configuration space (C-space)**. The configuration of a robot is represented by a point in its C-space.

In this chapter we study the C-space and degrees of freedom of general robots.

¹ Compare with trying to represent the configuration of a soft object like a pillow.

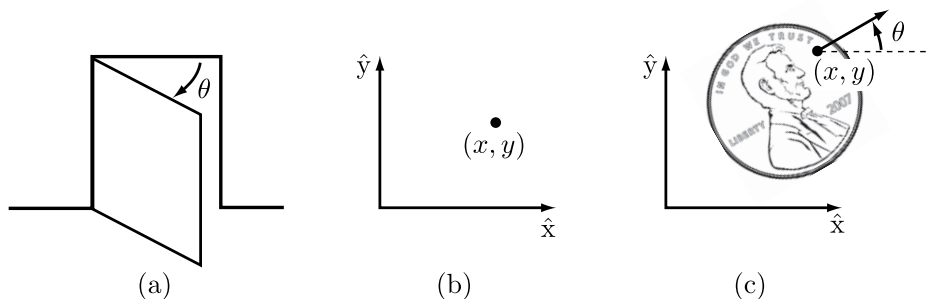


Figure 2.1 (a) The configuration of a door is described by the angle θ . (b) The configuration of a point in a plane is described by coordinates (x, y) . (c) The configuration of a coin on a table is described by (x, y, θ) , where θ defines the direction in which Abraham Lincoln is looking.

Since our robots are constructed from rigid links, we examine first the degrees of freedom of a single rigid body, and then the degrees of freedom of general multi-link robots. Next we study the shape (or topology) and geometry of C-spaces and their mathematical representation. The chapter concludes with a discussion of the C-space of a robot's end-effector, its **task space**. In the following chapter we study in more detail the mathematical representation of the C-space of a single rigid body.

2.1 Degrees of Freedom of a Rigid Body

Continuing with the example of the coin lying on the table, choose three points A , B , and C on the coin (Figure 2.2(a)). Once a coordinate frame $\hat{x}$ - $\hat{y}$ is attached to the plane,² the positions of these points in the plane are written (x_A, y_A) , (x_B, y_B) , and (x_C, y_C) . If the points could be placed independently anywhere in the plane, the coin would have six degrees of freedom – two for each of the three points. But, according to the definition of a rigid body, the distance between point A and point B , denoted $d(A, B)$, is always constant regardless of where the coin is. Similarly, the distances $d(B, C)$ and $d(A, C)$ must be constant. The following equality constraints on the coordinates (x_A, y_A) , (x_B, y_B) , and (x_C, y_C) must therefore always be satisfied:

$$\begin{aligned} d(A, B) &= \sqrt{(x_A - x_B)^2 + (y_A - y_B)^2} = d_{AB}, \\ d(B, C) &= \sqrt{(x_B - x_C)^2 + (y_B - y_C)^2} = d_{BC}, \\ d(A, C) &= \sqrt{(x_A - x_C)^2 + (y_A - y_C)^2} = d_{AC}. \end{aligned}$$

To determine the number of degrees of freedom of the coin on the table, first

² The unit axes of coordinate frames are written with a hat, indicating they are unit vectors, and in a non-italic font, e.g., $\hat{x}$, $\hat{y}$, and $\hat{z}$.

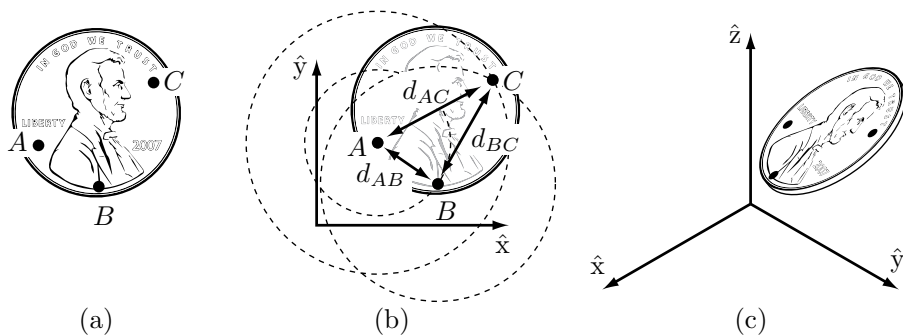


Figure 2.2 (a) Choosing three points fixed to the coin. (b) Once the location of A is chosen, B must lie on a circle of radius d_{AB} centered at A . Once the location of B is chosen, C must lie at the intersection of circles centered at A and B . Only one of these two intersections corresponds to the “heads up” configuration. (c) The configuration of a coin in three-dimensional space is given by the three coordinates of A , two angles to the point B on the sphere of radius d_{AB} centered at A , and one angle to the point C on the circle defined by the intersection of the a sphere centered at A and a sphere centered at B .

choose the position of point A in the plane (Figure 2.2(b)). We may choose it to be anything we want, so we have two degrees of freedom to specify, namely (x_A, y_A) . Once (x_A, y_A) is specified, the constraint $d(A, B) = d_{AB}$ restricts the choice of (x_B, y_B) to those points on the circle of radius d_{AB} centered at A . A point on this circle can be specified by a single parameter, e.g., the angle specifying the location of B on the circle centered at A . Let’s call this angle ϕ_{AB} and define it to be the angle that the vector $\vec{AB}$ makes with the $\hat{x}$ -axis.

Once we have chosen the location of point B , there are only two possible locations for C : at the intersections of the circle of radius d_{AC} centered at A and the circle of radius d_{BC} centered at B (Figure 2.2(b)). These two solutions correspond to heads or tails. In other words, once we have placed A and B and chosen heads or tails, the two constraints $d(A, C) = d_{AC}$ and $d(B, C) = d_{BC}$ eliminate the two apparent freedoms provided by (x_C, y_C) , and the location of C is fixed. The coin has exactly three degrees of freedom in the plane, which can be specified by (x_A, y_A, ϕ_{AB}) .

Suppose that we choose to specify the position of an additional point D on the coin. This introduces three additional constraints: $d(A, D) = d_{AD}$, $d(B, D) = d_{BD}$, and $d(C, D) = d_{CD}$. One of these constraints is **redundant**, i.e., it provides no new information; only two of the three constraints are independent. The two freedoms apparently introduced by the coordinates (x_D, y_D) are then immediately eliminated by these two independent constraints. The same would hold for any other newly chosen point on the coin, so that there is no need to consider additional points.

We have been applying the following general rule for determining the number

of degrees of freedom of a system:

$$\begin{aligned} \text{degrees of freedom} = & (\text{sum of freedoms of the points}) - \\ & (\text{number of independent constraints}). \end{aligned} \quad (2.1)$$

This rule can also be expressed in terms of the number of variables and independent equations that describe the system:

$$\begin{aligned} \text{degrees of freedom} = & (\text{number of variables}) - \\ & (\text{number of independent equations}). \end{aligned} \quad (2.2)$$

This general rule can also be used to determine the number of freedoms of a rigid body in three dimensions. For example, assume our coin is no longer confined to the table (Figure 2.2(c)). The coordinates for the three points A , B , and C are now given by (x_A, y_A, z_A) , (x_B, y_B, z_B) , and (x_C, y_C, z_C) , respectively. Point A can be placed freely (three degrees of freedom). The location of point B is subject to the constraint $d(A, B) = d_{AB}$, meaning it must lie on the sphere of radius d_{AB} centered at A . Thus we have $3 - 1 = 2$ freedoms to specify, which can be expressed as the latitude and longitude for the point on the sphere. Finally, the location of point C must lie at the intersection of spheres centered at A and B of radius d_{AC} and d_{BC} , respectively. In the general case the intersection of two spheres is a circle, and the location of point C can be described by an angle that parametrizes this circle. Point C therefore adds $3 - 2 = 1$ freedom. Once the position of point C is chosen, the coin is fixed in space.

In summary, a rigid body in three-dimensional space has six freedoms, which can be described by the three coordinates parametrizing point A , the two angles parametrizing point B , and one angle parametrizing point C , provided A , B , and C are noncollinear. Other representations for the configuration of a rigid body are discussed in Chapter 3.

We have just established that a rigid body moving in three-dimensional space, which we call a **spatial rigid body**, has six degrees of freedom. Similarly, a rigid body moving in a two-dimensional plane, which we henceforth call a **planar rigid body**, has three degrees of freedom. This latter result can also be obtained by considering the planar rigid body to be a spatial rigid body with six degrees of freedom but with the three independent constraints $z_A = z_B = z_C = 0$.

Since our robots consist of rigid bodies, Equation (2.1) can be expressed as follows:

$$\begin{aligned} \text{degrees of freedom} = & (\text{sum of freedoms of the bodies}) - \\ & (\text{number of independent constraints}). \end{aligned} \quad (2.3)$$

Equation (2.3) forms the basis for determining the degrees of freedom of general robots, which is the topic of the next section.

2.2 Degrees of Freedom of a Robot

Consider once again the door example of Figure 2.1(a), consisting of a single rigid body connected to a wall by a hinge joint. From the previous section we know that the door has only one degree of freedom, conveniently represented by the hinge joint angle θ . Without the hinge joint, the door would be free to move in three-dimensional space and would have six degrees of freedom. By connecting the door to the wall via the hinge joint, five independent constraints are imposed on the motion of the door, leaving only one independent coordinate (θ). Alternatively, the door can be viewed from above and regarded as a planar body, which has three degrees of freedom. The hinge joint then imposes two independent constraints, again leaving only one independent coordinate (θ). The door's C-space is represented by some range in the interval $[0, 2\pi)$ over which θ is allowed to vary.

In both cases the joints constrain the motion of the rigid body, thus reducing the overall degrees of freedom. This observation suggests a formula for determining the number of degrees of freedom of a robot, simply by counting the number of rigid bodies and joints. In this section we derive precisely such a formula, called Grübler's formula, for determining the number of degrees of freedom of planar and spatial robots.

2.2.1 Robot Joints

Figure 2.3 illustrates the basic joints found in typical robots. Every joint connects exactly two links; joints that simultaneously connect three or more links are not allowed. The **revolute joint** (R), also called a hinge joint, allows rotational motion about the joint axis. The **prismatic joint** (P), also called a sliding or linear joint, allows translational (or rectilinear) motion along the direction of the joint axis. The **helical joint** (H), also called a screw joint, allows simultaneous rotation and translation about a screw axis. Revolute, prismatic, and helical joints all have one degree of freedom.

Joints can also have multiple degrees of freedom. The **cylindrical joint** (C) has two degrees of freedom and allows independent translations and rotations about a single fixed joint axis. The **universal joint** (U) is another two-degree-of-freedom joint that consists of a pair of revolute joints arranged so that their joint axes are orthogonal. The **spherical joint** (S), also called a ball-and-socket joint, has three degrees of freedom and functions much like our shoulder joint.

A joint can be viewed as providing freedoms to allow one rigid body to move relative to another. It can also be viewed as providing constraints on the possible motions of the two rigid bodies it connects. For example, a revolute joint can be viewed as allowing one freedom of motion between two rigid bodies in space, or it can be viewed as providing five constraints on the motion of one rigid body relative to the other. Generalizing, the number of degrees of freedom of a rigid body (three for planar bodies and six for spatial bodies) minus the number of

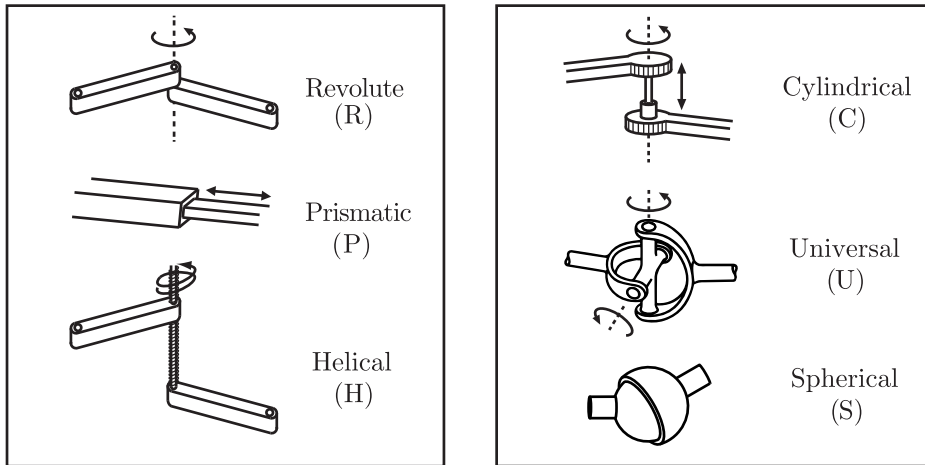


Figure 2.3 Typical robot joints.

Joint type	dof f	Constraints c between two planar rigid bodies	Constraints c between two spatial rigid bodies
Revolute (R)	1	2	5
Prismatic (P)	1	2	5
Helical (H)	1	N/A	5
Cylindrical (C)	2	N/A	4
Universal (U)	2	N/A	4
Spherical (S)	3	N/A	3

Table 2.1 The number of degrees of freedom f and constraints c provided by common joints.

constraints provided by a joint must equal the number of freedoms provided by that joint.

The freedoms and constraints provided by the various joint types are summarized in Table 2.1.

2.2.2 Grübler's Formula

The number of degrees of freedom of a mechanism with links and joints can be calculated using **Grübler's formula**, which is an expression of Equation (2.3).

Proposition 2.2 Consider a mechanism consisting of N links, where ground is also regarded as a link. Let J be the number of joints, m be the number of degrees of freedom of a rigid body ($m = 3$ for planar mechanisms and $m = 6$ for spatial mechanisms), f_i be the number of freedoms provided by joint i , and c_i be the number of constraints provided by joint i , where $f_i + c_i = m$ for all i . Then

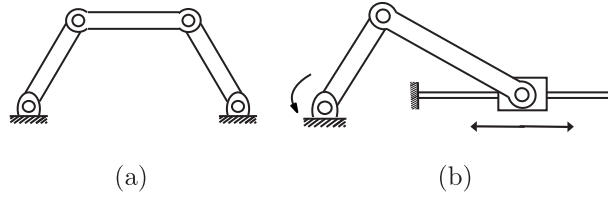


Figure 2.4 (a) Four-bar linkage. (b) Slider-crank mechanism.

Grübler's formula for the number of degrees of freedom of the robot is

$$\begin{aligned}
 \text{dof} &= \underbrace{m(N-1)}_{\text{rigid body freedoms}} - \underbrace{\sum_{i=1}^J c_i}_{\text{joint constraints}} \\
 &= m(N-1) - \sum_{i=1}^J (m - f_i) \\
 &= m(N-1-J) + \sum_{i=1}^J f_i.
 \end{aligned} \tag{2.4}$$

This formula holds only if all joint constraints are independent. If they are not independent then the formula provides a lower bound on the number of degrees of freedom.

Below we apply Grübler's formula to several planar and spatial mechanisms. We distinguish between two types of mechanism: **open-chain mechanisms** (also known as **serial mechanisms**) and **closed-chain mechanisms**. A closed-chain mechanism is any mechanism that has a closed loop. A person standing with both feet on the ground is an example of a closed-chain mechanism, since a closed loop can be traced from the ground, through the right leg, through the waist, through the left leg, and back to ground (recall that the ground itself is a link). An open-chain mechanism is any mechanism without a closed loop; an example is your arm when your hand is allowed to move freely in space.

Example 2.3 (Four-bar linkage and slider-crank mechanism) The planar four-bar linkage shown in Figure 2.4(a) consists of four links (one of them ground) arranged in a single closed loop and connected by four revolute joints. Since all the links are confined to move in the same plane, we have $m = 3$. Substituting $N = 4$, $J = 4$, and $f_i = 1$, $i = 1, \dots, 4$, into Grübler's formula, we see that the four-bar linkage has one degree of freedom.

The slider-crank closed-chain mechanism of Figure 2.4(b) can be analyzed in two ways: (i) the mechanism consists of three revolute joints and one prismatic joint ($J = 4$ and each $f_i = 1$) and four links ($N = 4$, including the ground link), or (ii) the mechanism consists of two revolute joints ($f_i = 1$) and one RP joint (the RP joint is a concatenation of a revolute and prismatic joint, so that $f_i = 2$)

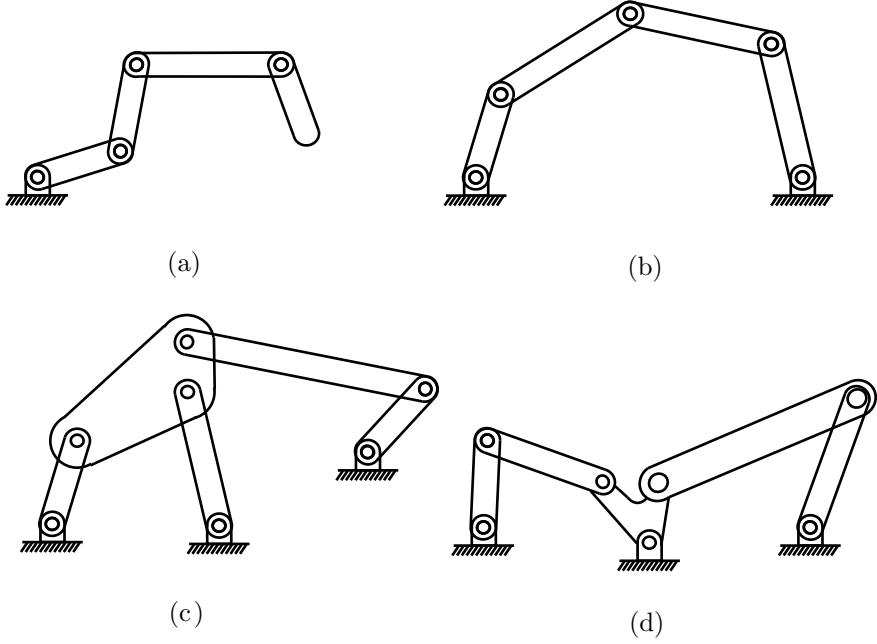


Figure 2.5 (a) A k -link planar serial chain. (b) Five-bar planar linkage. (c) Stephenson six-bar linkage. (d) Watt six-bar linkage.

and three links ($N = 3$; remember that each joint connects precisely two bodies). In both cases the mechanism has one degree of freedom.

Example 2.4 (Some classical planar mechanisms) Let us now apply Grübler's formula to several classical planar mechanisms. The k -link planar serial chain of revolute joints in Figure 2.5(a) (called a k R robot for its k revolute joints) has $N = k + 1$ links (k links plus ground), and $J = k$ joints, and, since all the joints are revolute, $f_i = 1$ for all i . Therefore,

$$\text{dof} = 3((k + 1) - 1 - k) + k = k$$

as expected. For the planar five-bar linkage of Figure 2.5(b), $N = 5$ (four links plus ground), $J = 5$, and since all joints are revolute, each $f_i = 1$. Therefore,

$$\text{dof} = 3(5 - 1 - 5) + 5 = 2.$$

For the Stephenson six-bar linkage of Figure 2.5(c), we have $N = 6$, $J = 7$, and $f_i = 1$ for all i , so that

$$\text{dof} = 3(6 - 1 - 7) + 7 = 1.$$

Finally, for the Watt six-bar linkage of Figure 2.5(d), we have $N = 6$, $J = 7$,

and $f_i = 1$ for all i , so that, like the Stephenson six-bar linkage,

$$\text{dof} = 3(6 - 1 - 7) + 7 = 1.$$

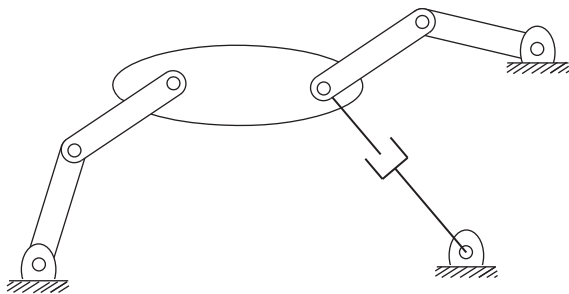


Figure 2.6 A planar mechanism with two overlapping joints.

Example 2.5 (A planar mechanism with overlapping joints) The planar mechanism illustrated in Figure 2.6 has three links that meet at a single point on the right of the large link. Recalling that a joint by definition connects exactly two links, the joint at this point of intersection should not be regarded as a single revolute joint. Rather, it is correctly interpreted as two revolute joints overlapping each other. Again, there is more than one way to derive the number of degrees of freedom of this mechanism using Grübler's formula: (i) The mechanism consists of eight links ($N = 8$), eight revolute joints, and one prismatic joint. Substituting into Grübler's formula yields

$$\text{dof} = 3(8 - 1 - 9) + 9(1) = 3.$$

(ii) Alternatively, the lower-right revolute–prismatic joint pair can be regarded as a single two-dof joint. In this case the number of links is $N = 7$, with seven revolute joints, and a single two-dof revolute–prismatic pair. Substituting into Grübler's formula yields

$$\text{dof} = 3(7 - 1 - 8) + 7(1) + 1(2) = 3.$$

Example 2.6 (Redundant constraints and singularities) For the parallelogram linkage of Figure 2.7(a), $N = 5$, $J = 6$, and $f_i = 1$ for each joint. From Grübler's

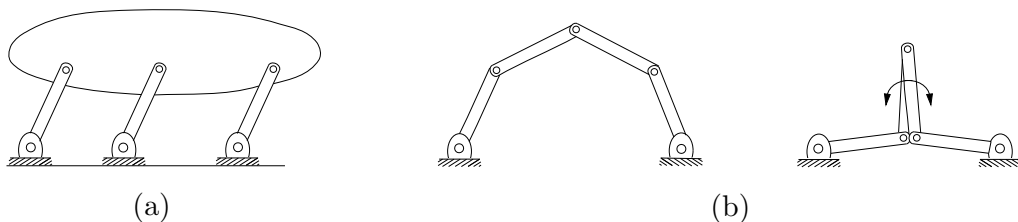


Figure 2.7 (a) A parallelogram linkage. (b) The five-bar linkage in a regular and singular configuration.

formula, the number of degrees of freedom is $3(5 - 1 - 6) + 6 = 0$. A mechanism with zero degrees of freedom is by definition a rigid structure. It is clear from examining the figure, though, that the mechanism can in fact move with one degree of freedom. Indeed, any one of the three parallel links, with its two joints, has no effect on the motion of the mechanism, so we should have calculated $\text{dof} = 3(4 - 1 - 4) + 4 = 1$. In other words, the constraints provided by the joints are not independent, as required by Grübler's formula.

A similar situation arises for the two-dof planar five-bar linkage of Figure 2.7(b). If the two joints connected to ground are locked at some fixed angle, the five-bar linkage should then become a rigid structure. However, if the two middle links are the same length and overlap each other, as illustrated in Figure 2.7(b), these overlapping links can rotate freely about the two overlapping joints. Of course, the link lengths of the five-bar linkage must meet certain specifications in order for such a configuration to even be possible. Also note that if a different pair of joints is locked in place, the mechanism does become a rigid structure as expected.

Grübler's formula provides a lower bound on the degrees of freedom for cases like those just described. Configuration space singularities arising in closed chains are discussed in Chapter 7.

Example 2.7 (Delta robot) The Delta robot of Figure 2.8 consists of two platforms – the lower one mobile, the upper one stationary – connected by three legs. Each leg contains a parallelogram closed chain and consists of three revolute joints, four spherical joints, and five links. Adding the two platforms, there

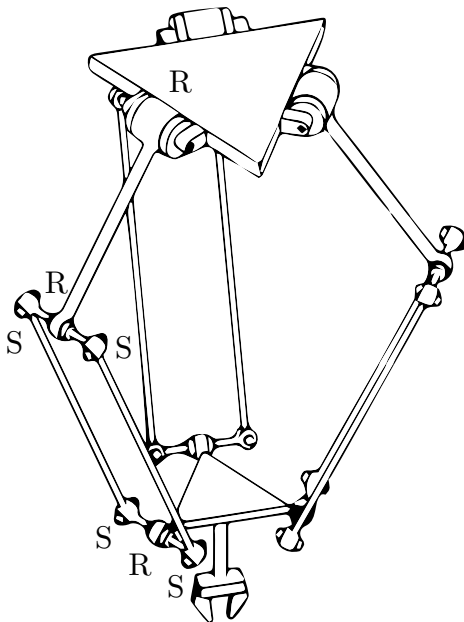


Figure 2.8 The Delta robot.

are $N = 17$ links and $J = 21$ joints (nine revolute and 12 spherical). By Grübler's formula,

$$\text{dof} = 6(17 - 1 - 21) + 9(1) + 12(3) = 15.$$

Of these 15 degrees of freedom, however, only three are visible at the end-effector on the moving platform. In fact, the parallelogram leg design ensures that the moving platform always remains parallel to the fixed platform, so that the Delta robot acts as an x - y - z Cartesian positioning device. The other 12 internal degrees of freedom are accounted for by torsion of the 12 links in the parallelograms (each of the three legs has four links in its parallelogram) about their long axes.

Example 2.8 (Stewart–Gough platform) The Stewart–Gough platform of Figure 1.1(b) consists of two platforms – the lower one stationary and regarded as ground, the upper one mobile – connected by six universal–prismatic–spherical (UPS) legs. The total number of links is 14 ($N = 14$). There are six universal joints (each with two degrees of freedom, $f_i = 2$), six prismatic joints (each with a single degree of freedom, $f_i = 1$), and six spherical joints (each with three degrees of freedom, $f_i = 3$). The total number of joints is 18. Substituting these values into Grübler's formula with $m = 6$ yields

$$\text{dof} = 6(14 - 1 - 18) + 6(1) + 6(2) + 6(3) = 6.$$

In some versions of the Stewart–Gough platform the six universal joints are replaced by spherical joints. By Grübler's formula this mechanism has 12 degrees of freedom; replacing each universal joint by a spherical joint introduces an extra degree of freedom in each leg, allowing torsional rotations about the leg axis. Note, however, that this torsional rotation has no effect on the motion of the mobile platform.

The Stewart–Gough platform is a popular choice for car and airplane cockpit simulators, as the platform moves with the full six degrees of freedom of motion of a rigid body. On the one hand, the parallel structure means that each leg needs to support only a fraction of the weight of the payload. On the other hand, this structure also limits the range of translational and rotational motion of the platform relative to the range of motion of the end-effector of a six-dof open chain.

2.3 Configuration Space: Topology and Representation

2.3.1 Configuration Space Topology

Until now we have been focusing on one important aspect of a robot's C-space – its dimension, or the number of degrees of freedom. However, the *shape* of the space is also important.

Consider a point moving on the surface of a sphere. The point's C-space is two dimensional, as the configuration can be described by two coordinates, latitude

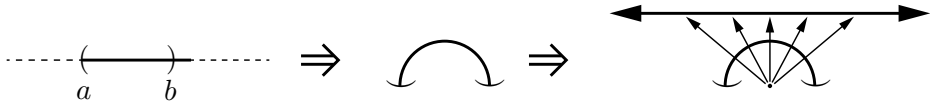


Figure 2.9 An open interval of the real line, denoted (a, b) , can be deformed to an open semicircle. This open semicircle can then be deformed to the real line by the mapping illustrated: beginning from a point at the center of the semicircle, draw a ray that intersects the semicircle and then a line above the semicircle. These rays show that every point of the semicircle can be stretched to exactly one point on the line, and vice versa. Thus an open interval can be continuously deformed to a line, so an open interval and a line are topologically equivalent.

and longitude. As another example, a point moving on a plane also has a two-dimensional C-space, with coordinates (x, y) . While both a plane and the surface of a sphere are two dimensional, clearly they do not have the same shape – the plane extends infinitely while the sphere wraps around.

Unlike the plane, a larger sphere has the same shape as the original sphere, in that it wraps around in the same way. Only its size is different. For that matter, an oval-shaped American football also wraps around similarly to a sphere. The only difference between a football and a sphere is that the football has been stretched in one direction.

The idea that the two-dimensional surfaces of a small sphere, a large sphere, and a football all have the same kind of shape, which is different from the shape of a plane, is expressed by the **topology** of the surfaces. We do not attempt a rigorous treatment in this book,³ but we say that two spaces are **topologically equivalent** if one can be continuously deformed into the other without cutting or gluing. A sphere can be deformed into a football simply by stretching, without cutting or gluing, so those two spaces are topologically equivalent. You cannot turn a sphere into a plane without cutting it, however, so a sphere and a plane are not topologically equivalent.

Topologically distinct one-dimensional spaces include the circle, the line, and a closed interval of the line. The circle is written mathematically as S or S^1 , a one-dimensional “sphere.” The line can be written as $\mathbb{E}$ or $\mathbb{E}^1$, indicating a one-dimensional Euclidean (or “flat”) space. Since a point in $\mathbb{E}^1$ is usually represented by a real number (after choosing an origin and a length scale), it is often written as $\mathbb{R}$ or $\mathbb{R}^1$ instead. A closed interval of the line, which contains its endpoints, can be written $[a, b] \subset \mathbb{R}^1$. (An open interval (a, b) does not include the endpoints a and b and is topologically equivalent to a line, since the open interval can be stretched to a line, as shown in Figure 2.9. A closed interval is not topologically equivalent to a line, since a line does not contain endpoints.)

In higher dimensions, $\mathbb{R}^n$ is the n -dimensional Euclidean space and S^n is the

³ For those familiar with concepts in topology, all the spaces we consider can be viewed as embedded in a higher-dimensional Euclidean space, inheriting the Euclidean topology of that space.

n -dimensional surface of a sphere in $(n + 1)$ -dimensional space. For example, S^2 is the two-dimensional surface of a sphere in three-dimensional space.

Note that the topology of a space is a fundamental property of the space itself and *is independent of how we choose coordinates to represent points in the space*. For example, to represent a point on a circle, we could refer to the point by the angle θ from the center of the circle to the point, relative to a chosen zero angle. Or, we could choose a reference frame with its origin at the center of the circle and represent the point by the two coordinates (x, y) subject to the constraint $x^2 + y^2 = 1$. No matter what our choice of coordinates is, the space itself does not change.

Some C-spaces can be expressed as the **Cartesian product** of two or more spaces of lower dimension; that is, points in such a C-space can be represented as the union of the representations of points in the lower-dimensional spaces. For example:

- The C-space of a rigid body in the plane can be written as $\mathbb{R}^2 \times S^1$, since the configuration can be represented as the concatenation of the coordinates (x, y) representing $\mathbb{R}^2$ and an angle θ representing S^1 .
- The C-space of a PR robot arm can be written $\mathbb{R}^1 \times S^1$. (We will occasionally ignore joint limits, i.e., bounds on the travel of the joints, when expressing the topology of the C-space; with joint limits, the C-space is the Cartesian product of two closed intervals of the line.)
- The C-space of a 2R robot arm can be written $S^1 \times S^1 = T^2$, where T^n is the n -dimensional surface of a torus in an $(n + 1)$ -dimensional space. (See Table 2.2.) Note that $S^1 \times S^1 \times \dots \times S^1$ (n copies of S^1) is equal to T^n , not S^n ; for example, a sphere S^2 is not topologically equivalent to a torus T^2 .
- The C-space of a planar rigid body (e.g., the chassis of a mobile robot) with a 2R robot arm can be written as $\mathbb{R}^2 \times S^1 \times T^2 = \mathbb{R}^2 \times T^3$.
- As we saw in Section 2.1 when we counted the degrees of freedom of a rigid body in three dimensions, the configuration of a rigid body can be described by a point in $\mathbb{R}^3$, plus a point on a two-dimensional sphere S^2 , plus a point on a one-dimensional circle S^1 , giving a total C-space of $\mathbb{R}^3 \times S^2 \times S^1$.

2.3.2 Configuration Space Representation

To perform computations, we must have a numerical **representation** of the space, consisting of a set of real numbers. We are familiar with this idea from linear algebra – a vector is a natural way to represent a point in a Euclidean space. It is important to keep in mind that the representation of a space involves a choice, and therefore it is not as fundamental as the topology of the space, which is independent of the representation. For example, the same point in a three-dimensional space can have different coordinate representations depending on the choice of reference frame (the origin and the direction of the coordinate

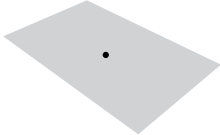
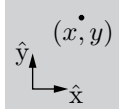
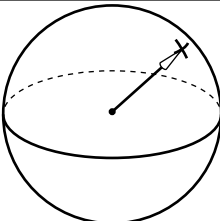
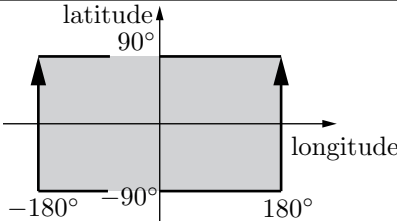
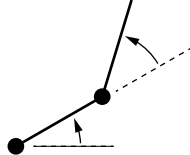
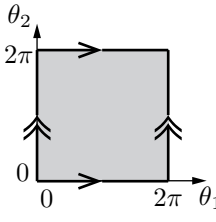
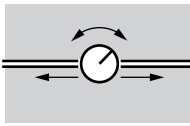
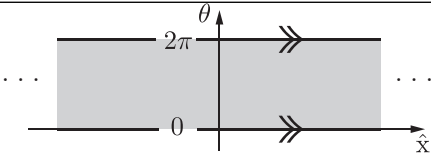
system	topology	sample representation
 point on a plane	 $\mathbb{E}^2$	 $\mathbb{R}^2$
 spherical pendulum	 S^2	 $[-180^\circ, 180^\circ) \times [-90^\circ, 90^\circ]$
 2R robot arm	 $T^2 = S^1 \times S^1$	 $[0, 2\pi) \times [0, 2\pi)$
 rotating sliding knob	 $\mathbb{E}^1 \times S^1$	 $\mathbb{R}^1 \times [0, 2\pi)$

Table 2.2 Four topologically different two-dimensional C-spaces and example coordinate representations. In the latitude–longitude representation of the sphere, the latitudes -90° and 90° each correspond to a single point (the south pole and the north pole, respectively), and the longitude parameter wraps around at 180° and -180° ; the edges with arrows are glued together. Similarly, the coordinate representations of the torus and cylinder wrap around at the edges marked with corresponding arrows.

axes) and the choice of length scale, but the topology of the underlying space is the same regardless of these choices.

While it is natural to choose a reference frame and length scale and to use a vector to represent points in a Euclidean space, representing a point on a curved space, such as a sphere, is less obvious. One solution for a sphere is to use latitude and longitude coordinates. A choice of n coordinates, or parameters, to represent an n -dimensional space is called an **explicit parametrization** of the space. Such an explicit parametrization is valid for a particular range of the parameters (e.g., $[-90^\circ, 90^\circ]$ for latitude and $[-180^\circ, 180^\circ)$ for longitude for

a sphere, where, on Earth, negative values correspond to “south” and “west,” respectively).

The latitude–longitude representation of a sphere is unsatisfactory if you are walking near the North Pole (where the latitude equals 90°) or South Pole (where the latitude equals -90°), where taking a very small step can result in a large change in the coordinates. The North and South Poles are **singularities** of the representation, and the existence of singularities is a result of the fact that a sphere does not have the same topology as a plane, i.e., the space of the two real numbers that we have chosen to represent the sphere (latitude and longitude). The location of these singularities has nothing to do with the sphere itself, which looks the same everywhere, and everything to do with the chosen representation of it. Singularities of the parametrization are particularly problematic when representing velocities as the time rate of change of coordinates, since these representations may tend to infinity near singularities even if the point on the sphere is moving at a constant speed $\sqrt{\dot{x}^2 + \dot{y}^2 + \dot{z}^2}$ (which is what the speed would be had you represented the point as (x, y, z) instead).

If you can assume that the configuration never approaches a singularity of the representation, you can ignore this issue. If you cannot make this assumption, there are two ways to overcome the problem.

- Use more than one **coordinate chart** on the space, where each coordinate chart is an explicit parametrization covering only a portion of the space such that, within each chart, there is no singularity. As the configuration representation approaches a singularity in one chart, e.g., the North or South Pole, you simply switch to another chart where the North and South Poles are far from singularities.

If we define a set of singularity-free coordinate charts that overlap each other and cover the entire space, like the two charts above, the charts are said to form an **atlas** of the space, much as an atlas of the Earth consists of several maps that together cover the Earth. An advantage of using an atlas of coordinate charts is that the representation always uses the minimum number of numbers. A disadvantage is the extra bookkeeping required to switch representations between coordinate charts to avoid singularities. (Note that Euclidean spaces can be covered by a single coordinate chart without singularities.)

- Use an **implicit representation** of the space instead of an explicit parametrization. An implicit representation views the n -dimensional space as embedded in a Euclidean space of more than n dimensions, just as a two-dimensional unit sphere can be viewed as a surface embedded in a three-dimensional Euclidean space. An implicit representation uses the coordinates of the higher-dimensional space (e.g., (x, y, z) in the three-dimensional space), but subjects these coordinates to constraints that reduce the number of degrees of freedom (e.g., $x^2 + y^2 + z^2 = 1$ for the unit sphere).

A disadvantage of this approach is that the representation has more

numbers than the number of degrees of freedom. An advantage is that there are no singularities in the representation – a point moving smoothly around the sphere is represented by a smoothly changing (x, y, z) , even at the North and South poles. A single representation is used for the whole sphere; multiple coordinate charts are not needed.

Another advantage is that while it may be very difficult to construct an explicit parametrization, or atlas, for a closed-chain mechanism, it is easy to find an implicit representation: the set of all joint coordinates subject to the **loop-closure equations** that define the closed loops (Section 2.4).

We will use implicit representations throughout the book, beginning in the next chapter. In particular, we use nine numbers, subject to six constraints, to represent the three orientation freedoms of a rigid body in space. This is called a **rotation matrix**. In addition to being singularity-free (unlike three-parameter representations such as roll–pitch–yaw angles⁴), the rotation matrix representation allows us to use linear algebra to perform computations such as rotating a rigid body or changing the reference frame in which the orientation of a rigid body is expressed.⁵

In summary, the non-Euclidean shape of many C-spaces motivates our use of implicit representations of C-space throughout this book. We return to this topic in the next chapter.

2.4 Configuration and Velocity Constraints

For robots containing one or more closed loops, usually an implicit representation is more easily obtained than an explicit parametrization. For example, consider the planar four-bar linkage of Figure 2.10, which has one degree of freedom. The fact that the four links always form a closed loop can be expressed by the following three equations:

$$\begin{aligned} L_1 \cos \theta_1 + L_2 \cos(\theta_1 + \theta_2) + \cdots + L_4 \cos(\theta_1 + \cdots + \theta_4) &= 0, \\ L_1 \sin \theta_1 + L_2 \sin(\theta_1 + \theta_2) + \cdots + L_4 \sin(\theta_1 + \cdots + \theta_4) &= 0, \\ \theta_1 + \theta_2 + \theta_3 + \theta_4 - 2\pi &= 0. \end{aligned}$$

These equations are obtained by viewing the four-bar linkage as a serial chain with four revolute joints in which (i) the tip of link L_4 always coincides with the origin and (ii) the orientation of link L_4 is always horizontal.

These equations are sometimes referred to as **loop-closure equations**. For

⁴ Roll–pitch–yaw angles and *Euler* angles use three parameters for the space of rotations $S^2 \times S^1$ (two for S^2 and one for S^1), and therefore are subject to singularities as discussed above.

⁵ Another singularity-free implicit representation of orientations, the unit quaternion, uses only four numbers subject to the constraint that the 4-vector be of unit length. In fact, this representation is a double covering of the set of orientations: for every orientation, there are two unit quaternions.

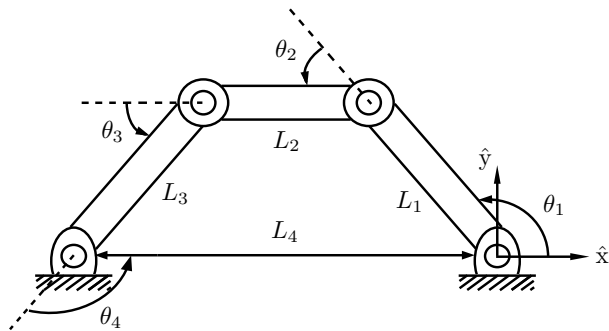


Figure 2.10 The four-bar linkage.

the four-bar linkage they are given by a set of three equations in four unknowns. The set of all solutions forms a one-dimensional curve in the four-dimensional joint space and constitutes the C-space.

In this book, when vectors are used in a linear algebra computation, they are generally treated as column vectors, e.g., $p = [1 \ 2 \ 3]^T$. When a computation is not imminent, however, we often think of a vector simply as an ordered list of variables, e.g., $p = (1, 2, 3)$.

Thus, for general robots containing one or more closed loops, the configuration space can be implicitly represented by the column vector $\theta = [\theta_1 \ \dots \ \theta_n]^T \in \mathbb{R}^n$ and loop-closure equations of the form

$$g(\theta) = \begin{bmatrix} g_1(\theta_1, \dots, \theta_n) \\ \vdots \\ g_k(\theta_1, \dots, \theta_n) \end{bmatrix} = 0, \quad (2.5)$$

a set of k independent equations, with $k \leq n$. Such constraints are known as **holonomic constraints**, ones that reduce the dimension of the C-space.⁶ The C-space can be viewed as a surface of dimension $n - k$ (assuming that all constraints are independent) embedded in $\mathbb{R}^n$.

Suppose that a closed-chain robot with loop-closure equations $g(\theta) = 0$, $g : \mathbb{R}^n \rightarrow \mathbb{R}^k$, is in motion, following the time trajectory $\theta(t)$. Differentiating both sides of $g(\theta(t)) = 0$ with respect to t , we obtain

$$\frac{d}{dt}g(\theta(t)) = 0; \quad (2.6)$$

⁶ Viewing a rigid body as a collection of points, the distance constraints between the points, as we saw earlier, can be viewed as holonomic constraints.

thus

$$\begin{bmatrix} \frac{\partial g_1}{\partial \theta_1}(\theta)\dot{\theta}_1 + \cdots + \frac{\partial g_1}{\partial \theta_n}(\theta)\dot{\theta}_n \\ \vdots \\ \frac{\partial g_k}{\partial \theta_1}(\theta)\dot{\theta}_1 + \cdots + \frac{\partial g_k}{\partial \theta_n}(\theta)\dot{\theta}_n \end{bmatrix} = 0.$$

This can be expressed as a matrix multiplying a column vector $[\dot{\theta}_1 \ \cdots \ \dot{\theta}_n]^T$:

$$\begin{bmatrix} \frac{\partial g_1}{\partial \theta_1}(\theta) & \cdots & \frac{\partial g_1}{\partial \theta_n}(\theta) \\ \vdots & \ddots & \vdots \\ \frac{\partial g_k}{\partial \theta_1}(\theta) & \cdots & \frac{\partial g_k}{\partial \theta_n}(\theta) \end{bmatrix} \begin{bmatrix} \dot{\theta}_1 \\ \vdots \\ \dot{\theta}_n \end{bmatrix} = 0,$$

which we can write as

$$\frac{\partial g}{\partial \theta}(\theta)\dot{\theta} = 0. \quad (2.7)$$

Here, the joint-velocity vector $\dot{\theta}_i$ denotes the derivative of θ_i with respect to time t , $\partial g(\theta)/\partial \theta \in \mathbb{R}^{k \times n}$, and $\theta, \dot{\theta} \in \mathbb{R}^n$. The constraints (2.7) can be written

$$A(\theta)\dot{\theta} = 0, \quad (2.8)$$

where $A(\theta) \in \mathbb{R}^{k \times n}$. Velocity constraints of this form are called **Pfaffian constraints**. For the case of $A(\theta) = \partial g(\theta)/\partial \theta$, one could regard $g(\theta)$ as being the “integral” of $A(\theta)$; for this reason, holonomic constraints of the form $g(\theta) = 0$ are also called **integrable constraints** – the velocity constraints that they imply can be integrated to give equivalent configuration constraints.

We now consider another class of Pfaffian constraints that are fundamentally different from the holonomic type. To illustrate this with a concrete example, consider an upright coin of radius r rolling on a plane as shown in Figure 2.11. The configuration of the coin is given by the contact point (x, y) on the plane, the steering angle ϕ , and the angle of rotation θ . The C-space of the coin is therefore $\mathbb{R}^2 \times T^2$, where T^2 is the two-dimensional torus parametrized by the angles ϕ and θ . This C-space is four dimensional.

We now express, in mathematical form, the fact that the coin rolls without slipping. The coin must always roll in the direction indicated by $(\cos \phi, \sin \phi)$, with forward speed $r\dot{\theta}$:

$$\begin{bmatrix} \dot{x} \\ \dot{y} \end{bmatrix} = r\dot{\theta} \begin{bmatrix} \cos \phi \\ \sin \phi \end{bmatrix}. \quad (2.9)$$

Collecting the four C-space coordinates into a single vector $q = [q_1 \ q_2 \ q_3 \ q_4]^T = [x \ y \ \phi \ \theta]^T \in \mathbb{R}^2 \times T^2$, the above no-slip rolling constraint can then be expressed in the form

$$\begin{bmatrix} 1 & 0 & 0 & -r \cos q_3 \\ 0 & 1 & 0 & -r \sin q_3 \end{bmatrix} \dot{q} = 0. \quad (2.10)$$

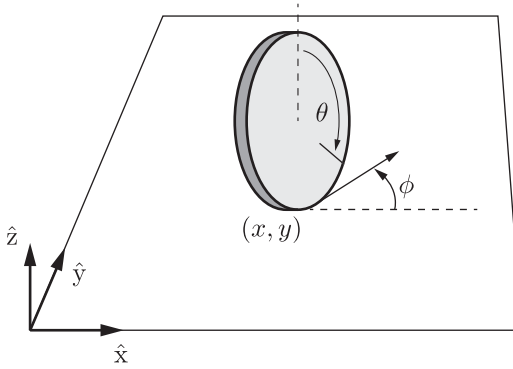


Figure 2.11 A coin rolling on a plane without slipping.

These are Pfaffian constraints of the form $A(q)\dot{q} = 0$, $A(q) \in \mathbb{R}^{2 \times 4}$.

These constraints are not integrable; that is, for the $A(q)$ given in (2.10), there does not exist a differentiable function $g : \mathbb{R}^4 \rightarrow \mathbb{R}^2$ such that $\partial g(q)/\partial q = A(q)$. If this were not the case then there would have to exist a differentiable $g_1(q)$ that satisfied the following four equalities:

$$\begin{aligned} \partial g_1(q)/\partial q_1 &= 1 & \longrightarrow & g_1(q) = q_1 + h_1(q_2, q_3, q_4) \\ \partial g_1(q)/\partial q_2 &= 0 & \longrightarrow & g_1(q) = h_2(q_1, q_3, q_4) \\ \partial g_1(q)/\partial q_3 &= 0 & \longrightarrow & g_1(q) = h_3(q_1, q_2, q_4) \\ \partial g_1(q)/\partial q_4 &= -r \cos q_3 & \longrightarrow & g_1(q) = -rq_4 \cos q_3 + h_4(q_1, q_2, q_3), \end{aligned}$$

for some h_i , $i = 1, \dots, 4$, differentiable in each of its variables. By inspection it should be clear that no such $g_1(q)$ exists. Similarly, it can be shown that $g_2(q)$ does not exist, so that the constraint (2.10) is nonintegrable. A Pfaffian constraint that is nonintegrable is called a **nonholonomic constraint**. Such constraints reduce the dimension of the feasible velocities of the system but do not reduce the dimension of the reachable C-space. The rolling coin can reach any point in its four-dimensional C-space despite the two constraints on its velocity.⁷ See Exercise 2.30.

In a number of robotics contexts nonholonomic constraints arise that involve the conservation of momentum and rolling without slipping, e.g., wheeled vehicle kinematics and grasp contact kinematics. We examine nonholonomic constraints in greater detail in our study of wheeled mobile robots in Chapter 13.

2.5 Task Space and Workspace

We now introduce two more concepts relating to the configuration of a robot: the task space and the workspace. Both relate to the configuration of the end-effector of a robot, not to the configuration of the entire robot.

⁷ Some texts define the number of degrees of freedom of a system to be the dimension of the feasible velocities, e.g., two for the rolling coin. We always refer to the dimension of the C-space as the number of degrees of freedom.

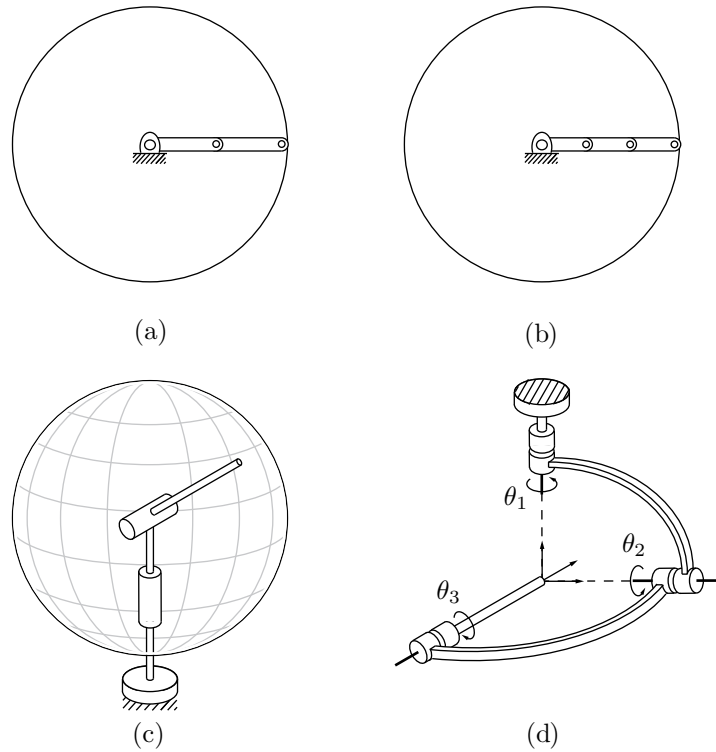


Figure 2.12 Examples of workspaces for various robots: (a) a planar 2R open chain; (b) a planar 3R open chain; (c) a spherical 2R open chain; (d) a 3R orienting mechanism.

The **task space** is a space in which the robot's task can be naturally expressed. For example, if the robot's task is to plot with a pen on a piece of paper, the task space would be $\mathbb{R}^2$. If the task is to manipulate a rigid body, a natural representation of the task space is the C-space of a rigid body, representing the position and orientation of a frame attached to the robot's end-effector. This is the default representation of task space. The decision of how to define the task space is driven by the task, independently of the robot.

The **workspace** is a specification of the configurations that the end-effector of the robot can reach. The definition of the workspace is primarily driven by the robot's structure, independently of the task.

Both the task space and the workspace involve a choice by the user; in particular, the user may decide that some freedoms of the end-effector (e.g., its orientation) do not need to be represented.

The task space and the workspace are distinct from the robot's C-space. A point in the task space or the workspace may be achievable by more than one robot configuration, meaning that the point is not a full specification of the

robot's configuration. For example, for an open-chain robot with seven joints, the six-dof position and orientation of its end-effector does not fully specify the robot's configuration.

Some points in the task space may not be reachable at all by the robot, such as some points on a chalkboard. By definition, however, all points in the workspace are reachable by at least one configuration of the robot.

Two mechanisms with different C-spaces may have the same workspace. For example, considering the end-effector to be the Cartesian tip of the robot (e.g., the location of a plotting pen) and ignoring orientations, the planar 2R open chain with links of equal length three (Figure 2.12(a)) and the planar 3R open chain with links of equal length two (Figure 2.12(b)) have the same workspace despite having different C-spaces.

Two mechanisms with the same C-space may also have different workspaces. For example, taking the end-effector to be the Cartesian tip of the robot and ignoring orientations, the 2R open chain of Figure 2.12(a) has a planar disk as its workspace, while the 2R open chain of Figure 2.12(c) has the surface of a sphere as its workspace.

Attaching a coordinate frame to the tip of the tool of the 3R open chain “wrist” mechanism of Figure 2.12(d), we see that the frame can achieve any orientation by rotating the joints but the Cartesian position of the tip is always fixed. This can be seen by noting that the three joint axes always intersect at the tip. For this mechanism, we would probably define the workspace to be the three-dof space of orientations of the frame, $S^2 \times S^1$, which is different from the C-space T^3 . The task space depends on the task; if the job is to point a laser pointer, then rotations about the axis of the laser beam are immaterial and the task space would be S^2 , the set of directions in which the laser can point.

Example 2.9 The SCARA robot of Figure 2.13 is an RRRP open chain that is widely used for tabletop pick-and-place tasks. The end-effector configuration is completely described by the four parameters (x, y, z, ϕ) , where (x, y, z) denotes the Cartesian position of the end-effector center point and ϕ denotes the orientation of the end-effector in the x - y -plane. Its task space would typically be defined as $\mathbb{R}^3 \times S^1$, and its workspace would typically be defined as the reachable points in (x, y, z) Cartesian space, since all orientations $\phi \in S^1$ can be achieved at all reachable points.

Example 2.10 A standard 6R industrial manipulator can be adapted to spray-painting applications as shown in Figure 2.14. The paint spray nozzle attached to the tip can be regarded as the end-effector. What is important to the task is the Cartesian position of the spray nozzle, together with the direction in which the spray nozzle is pointing; rotations about the nozzle axis (which points in the direction in which paint is being sprayed) do not matter. The nozzle configuration can therefore be described by five coordinates: (x, y, z) for the Cartesian position of the nozzle and spherical coordinates (θ, ϕ) to describe the direction in which

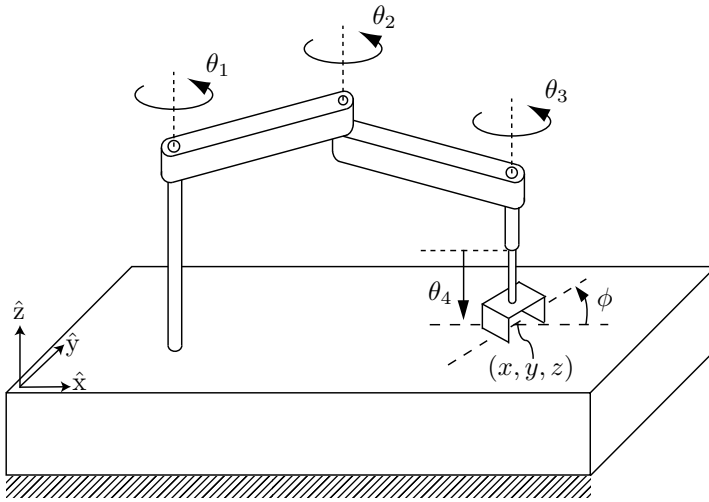


Figure 2.13 SCARA robot.

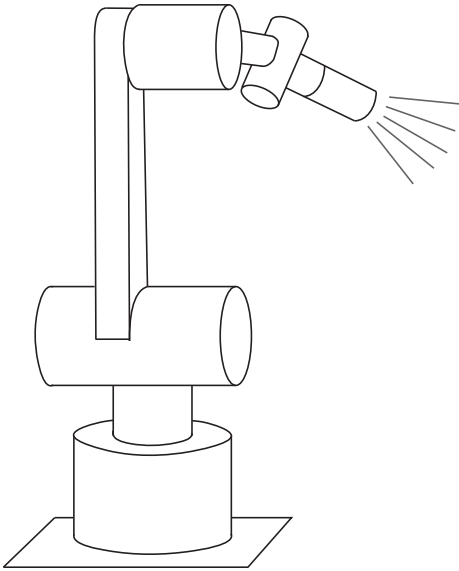


Figure 2.14 A spray-painting robot.

the nozzle is pointing. The task space can be written as $\mathbb{R}^3 \times S^2$. The workspace could be the reachable points in $\mathbb{R}^3 \times S^2$, or, to simplify visualization, the user could define the workspace to be the subset of $\mathbb{R}^3$ corresponding to the reachable Cartesian positions of the nozzle.

2.6 Summary

- A robot is mechanically constructed from links that are connected by various types of joint. The links are usually modeled as rigid bodies. An end-effector such as a gripper may be attached to some link of the robot. Actuators deliver forces and torques to the joints, thereby causing motion of the robot.
- The most widely used one-dof joints are the revolute joint, which allows rotation about the joint axis, and the prismatic joint, which allows translation in the direction of the joint axis. Some common two-dof joints include the cylindrical joint, which is constructed by serially connecting a revolute and prismatic joint, and the universal joint, which is constructed by orthogonally connecting two revolute joints. The spherical joint, also known as the ball-and-socket joint, is a three-dof joint whose function is similar to the human shoulder joint.
- The configuration of a rigid body is a specification of the location of all its points. For a rigid body moving in the plane, three independent parameters are needed to specify the configuration. For a rigid body moving in three-dimensional space, six independent parameters are needed to specify the configuration.
- The configuration of a robot is a specification of the configuration of all its links. The robot's configuration space is the set of all possible robot configurations. The dimension of the C-space is the number of degrees of freedom of a robot.
- The number of degrees of freedom of a robot can be calculated using Grübler's formula,

$$\text{dof} = m(N - 1 - J) + \sum_{i=1}^J f_i,$$

where $m = 3$ for planar mechanisms and $m = 6$ for spatial mechanisms, N is the number of links (including the ground link), J is the number of joints, and f_i is the number of degrees of freedom of joint i . If the constraints enforced by the joints are not independent then Grübler's formula provides a lower bound on the number of degrees of freedom.

- A robot's C-space can be parametrized explicitly or represented implicitly. For a robot with n degrees of freedom, an explicit parametrization uses n coordinates, the minimum necessary. An implicit representation involves m coordinates with $m \geq n$, with the m coordinates subject to $m - n$ constraint equations. With an implicit parametrization, a robot's C-space can be viewed as a surface of dimension n embedded in a space of higher dimension m .
- The C-space of an n -dof robot whose structure contains one or more closed loops can be implicitly represented using k loop-closure equations of the form $g(\theta) = 0$, where $\theta \in \mathbb{R}^m$ and $g : \mathbb{R}^m \rightarrow \mathbb{R}^k$. Such constraint equations are called holonomic constraints. Assuming that θ varies with time t , the

holonomic constraints $g(\theta(t)) = 0$ can be differentiated with respect to t to yield

$$\frac{\partial g}{\partial \theta}(\theta)\dot{\theta} = 0,$$

where $\partial g(\theta)/\partial \theta$ is a $k \times m$ matrix.

- A robot's motion can also be subject to velocity constraints of the form

$$A(\theta)\dot{\theta} = 0,$$

where $A(\theta)$ is a $k \times m$ matrix that cannot be expressed as the differential of some function $g(\theta)$. In other words, there does not exist any $g(\theta), g : \mathbb{R}^m \rightarrow \mathbb{R}^k$, such that

$$A(\theta) = \frac{\partial g}{\partial \theta}(\theta).$$

Such constraints are said to be nonholonomic constraints, or nonintegrable constraints. These constraints reduce the dimension of feasible velocities of the system but do not reduce the dimension of the reachable C-space. Nonholonomic constraints arise in robot systems subject to conservation of momentum or rolling without slipping.

- A robot's task space is a space in which the robot's task can be naturally expressed. A robot's workspace is a specification of the configurations that the end-effector of the robot can reach.

2.7 Notes and References

In the kinematics literature, structures that consist of links connected by joints are also called mechanisms or linkages. The number of degrees of freedom of a mechanism, also referred to as its mobility, is treated in most texts on mechanism analysis and design, e.g., Erdman and Sandor (1996) or McCarthy and Soh (2011). The notion of a robot's configuration space was first formulated in the context of motion planning by Lozano-Perez (1980); more recent and advanced treatments can be found in Latombe (1991), LaValle (2006), and Choset et al. (2005). As apparent from examples in this chapter, a robot's configuration space can be nonlinear and curved, as can its task space. Such spaces often have the mathematical structure of a differentiable manifold, which are the central objects of study in differential geometry. Some accessible introductions to differential geometry are Millman and Parker (1977), do Carmo (1976) and Boothby (2002).

2.8 Exercises

In the exercises below, if you are asked to “describe” a C-space, you should indicate its dimension and whatever you know about its topology (e.g., using $\mathbb{R}$, S , and T , as with the examples in Sections 2.3.1 and 2.3.2).

Exercise 2.5 Figure 2.15 shows a robot used for human arm rehabilitation. Determine the number of degrees of freedom of the chain formed by the human arm and the robot.

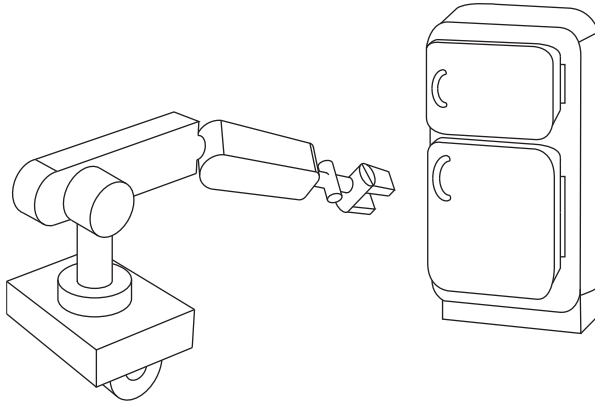


Figure 2.16 Mobile manipulator.

Exercise 2.6 The mobile manipulator of Figure 2.16 consists of a 6R arm and multi-fingered hand mounted on a mobile base with a single wheel. You can think of the wheeled base as the same as the rolling coin in Figure 2.11 – the wheel (and base) can spin together about an axis perpendicular to the ground, and the wheel rolls without slipping. The base always remains horizontal. (Left unstated are the means to keep the base horizontal and to spin the wheel and base about an axis perpendicular to the ground.)

- Ignoring the multi-fingered hand, describe the configuration space of the mobile manipulator.
- Now suppose that the robot hand rigidly grasps a refrigerator door handle and, with its wheel and base completely stationary, opens the door using only its arm. With the door open, how many degrees of freedom does the mechanism formed by the arm and open door have?
- A second identical mobile manipulator comes along, and after parking its mobile base, also rigidly grasps the refrigerator door handle. How many degrees of freedom does the mechanism formed by the two arms and the open refrigerator door have?

Exercise 2.7 Three identical SRS open-chain arms are grasping a common object, as shown in Figure 2.17.

- Find the number of degrees of freedom of this system.
- Suppose there are now a total of n such arms grasping the object. How many degrees of freedom does this system have?
- Suppose the spherical wrist joint in each of the n arms is now replaced by a universal joint. How many degrees of freedom does this system have?

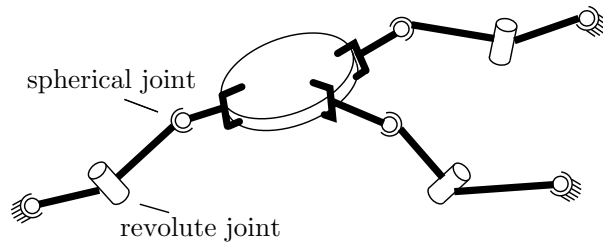


Figure 2.17 Three cooperating SRS arms grasping a common object.

Exercise 2.8 Consider a spatial parallel mechanism consisting of a moving plate connected to a fixed plate by n identical legs. For the moving plate to have six degrees of freedom, how many degrees of freedom should each leg have, as a function of n ? For example, if $n = 3$ then the moving plate and fixed plate are connected by three legs; how many degrees of freedom should each leg have for the moving plate to move with six degrees of freedom? Solve for arbitrary n .

Exercise 2.9 Use the planar version of Grübler's formula to determine the number of degrees of freedom of the mechanisms shown in Figure 2.18. Comment on whether your results agree with your intuition about the possible motions of these mechanisms.

Exercise 2.10 Use the planar version of Grübler's formula to determine the number of degrees of freedom of the mechanisms shown in Figure 2.19. Comment on whether your results agree with your intuition about the possible motions of these mechanisms.

Exercise 2.11 Use the spatial version of Grübler's formula to determine the number of degrees of freedom of the mechanisms shown in Figure 2.20. Comment on whether your results agree with your intuition about the possible motions of these mechanisms.

Exercise 2.12 Use the spatial version of Grübler's formula to determine the number of degrees of freedom of the mechanisms shown in Figure 2.21. Comment on whether your results agree with your intuition about the possible motions of these mechanisms.

Exercise 2.13 In the parallel mechanism shown in Figure 2.22, six legs of identical length are connected to a fixed and moving platform via spherical joints. Determine the number of degrees of freedom of this mechanism using Grübler's formula. Illustrate all possible motions of the upper platform.

Exercise 2.14 The $3 \times$ UPU platform of Figure 2.23 consists of two platforms – the lower one stationary, the upper one mobile – connected by three UPU legs.

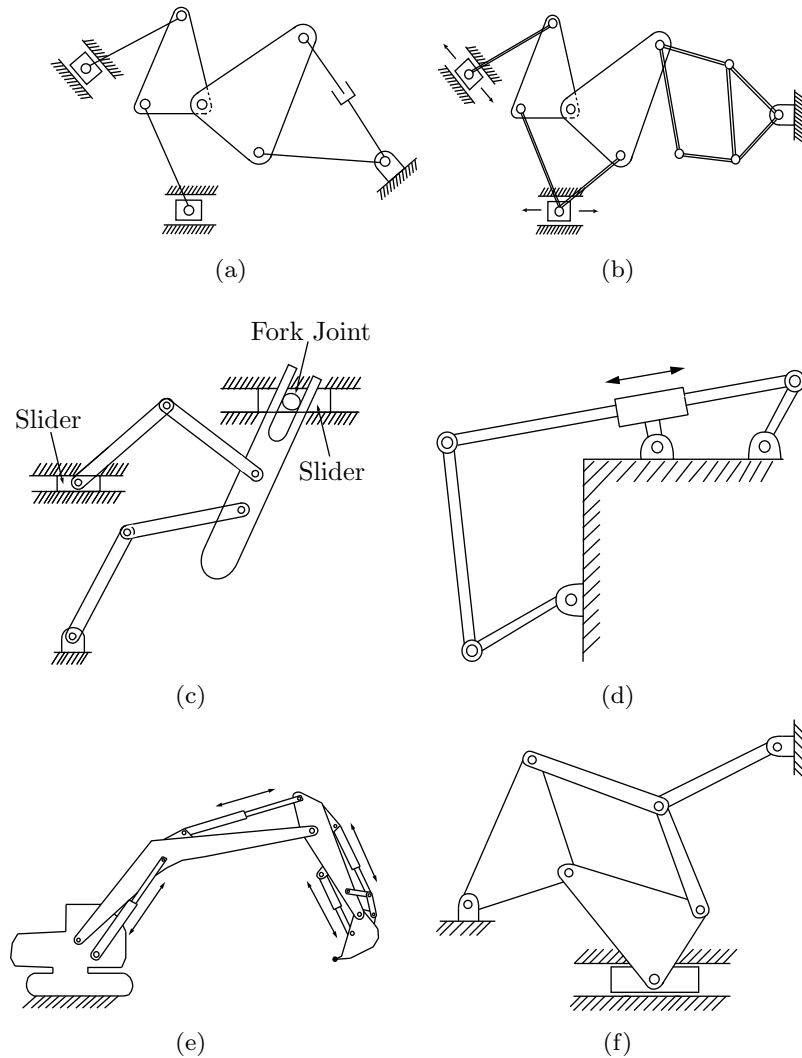


Figure 2.18 A first collection of planar mechanisms.

- (a) Using the spatial version of Grübler's formula, verify that it has three degrees of freedom.
- (b) Construct a physical model of the $3 \times \text{UPU}$ platform to see if it does indeed have three degrees of freedom. In particular, lock the three P joints in place; does the robot become a structure as predicted by Grübler's formula, or does it move?

Exercise 2.15 Consider the mechanisms of Figures 2.24(a) and 2.24(b).

- (a) The mechanism of Figure 2.24(a) consists of six identical squares arranged